

Week 9

Attention, Transformers

Nebius Academy



Plan

- Machine Translation Task, RNN Seq2Seq Architecture
- Attention Mechanism
- Transformer Architecture

Plan

In this module we will learn a mechanism of ***Attention***, and a neural network architecture based on this mechanism called ***Transformer***.

To better understand the nature of Attention and Transformer, we will first discuss a new natural language processing task — ***Machine Translation***.

We will see how neural models for this task evolved through time.

This will lead us to Attention and Transformer.

Machine Translation

Machine Translation

A sentence in English and its translation to German:

The cat is pretty $source = (x_1, x_2, \dots, x_n)$

Eine Katze ist hübsch $target = (y_1, y_2, \dots, y_n)$

Machine translation task is to find the most probable translation sequence given source sequence:

$$\widehat{target} = \operatorname{argmax}_{target} P(target | source, \theta)$$

Machine Translation

The cat is pretty

$source = (x_1, x_2, \dots, x_n)$

Eine Katze ist hübsch

$target = (y_1, y_2, \dots, y_n)$

$$P(target | source) = P(y_1 | source) \cdot P(y_2 | y_1, source) \\ \dots P(y_m | y_{m-1}, \dots y_1, source)$$



$$\widehat{target} = \underset{target}{\operatorname{argmax}} P(target | source, \theta)$$

Machine Translation

The task of Machine Translation is the task of
Conditional Language Modelling

$$P(target | source) = P(y_1 | source) \cdot P(y_2 | y_1, source) \\ \dots P(y_m | y_{m-1}, \dots y_1, source)$$



$$\widehat{target} = \underset{target}{argmax} P(target | source, \theta)$$

Machine Translation

Evolution of approaches to Machine Translation:

- Rule-based MT (1950th)
- Phrase-based/Statistical MT (1990th)
- Neural MT (2010th)

A nice article about the history of MT approaches: [link](#)

RNN for MT

We'll now build an RNN architecture for MT task.

As this is the task of ***Conditional Language Modelling***,
our network architecture will be similar to that for
Language Modelling task.

RNN for MT

A reminder of LM-RNN Architecture:

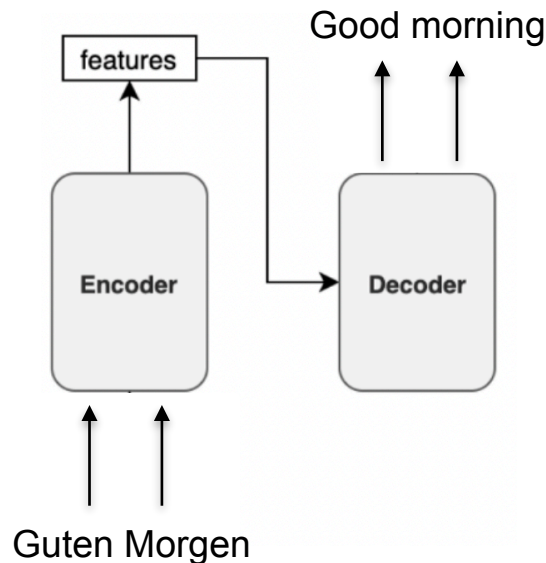
RNN for MT

Model idea:

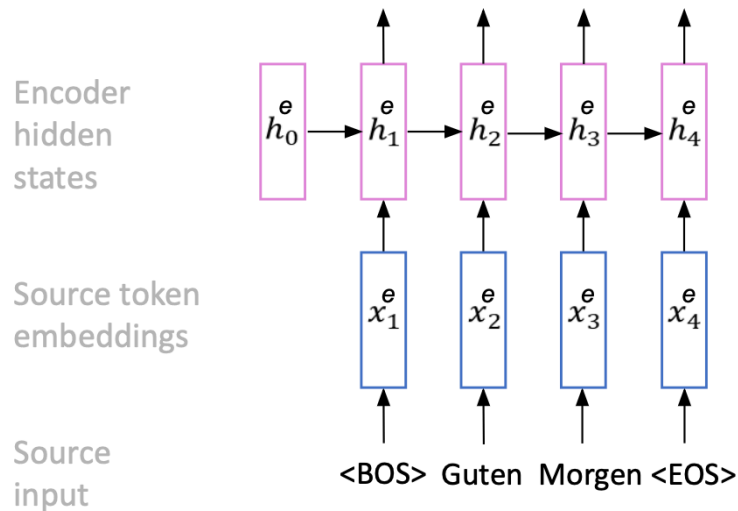
Our network will consist of 2 parts:

Encoder and Decoder.

- **Encoder** will read input sequence on source language and aggregate information from it;
- **Decoder** will generate output sequence on target language based on information given by Encoder.



RNN for MT



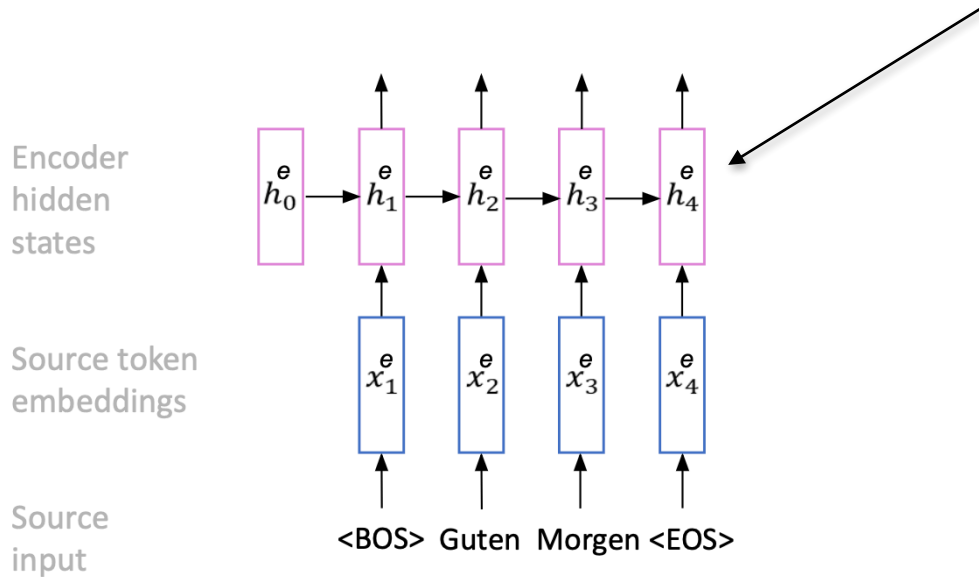
Encoder

Encoder is a neural network with Embedding and RNN layers

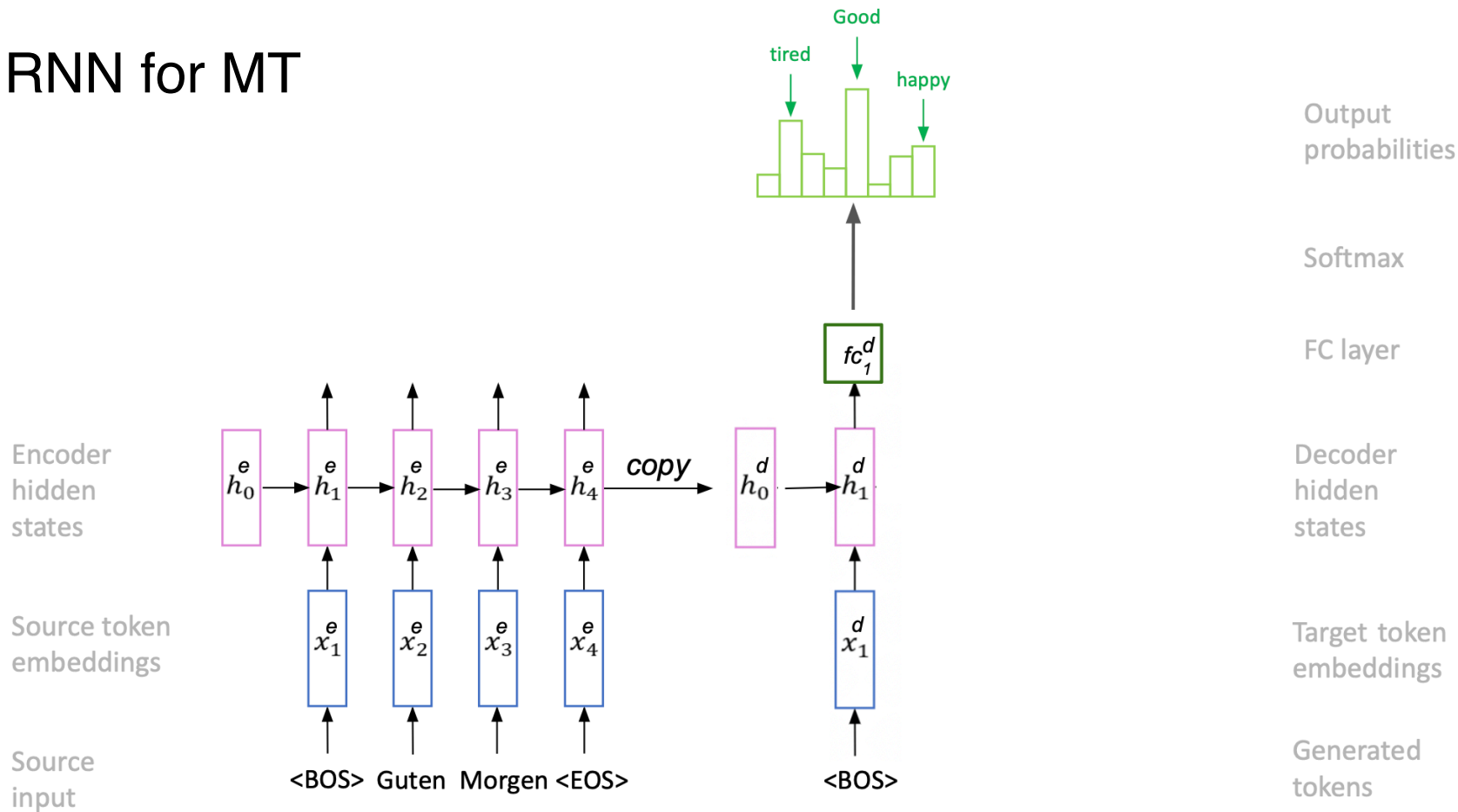
It “reads” input sequence token by token and updates its hidden states

RNN for MT

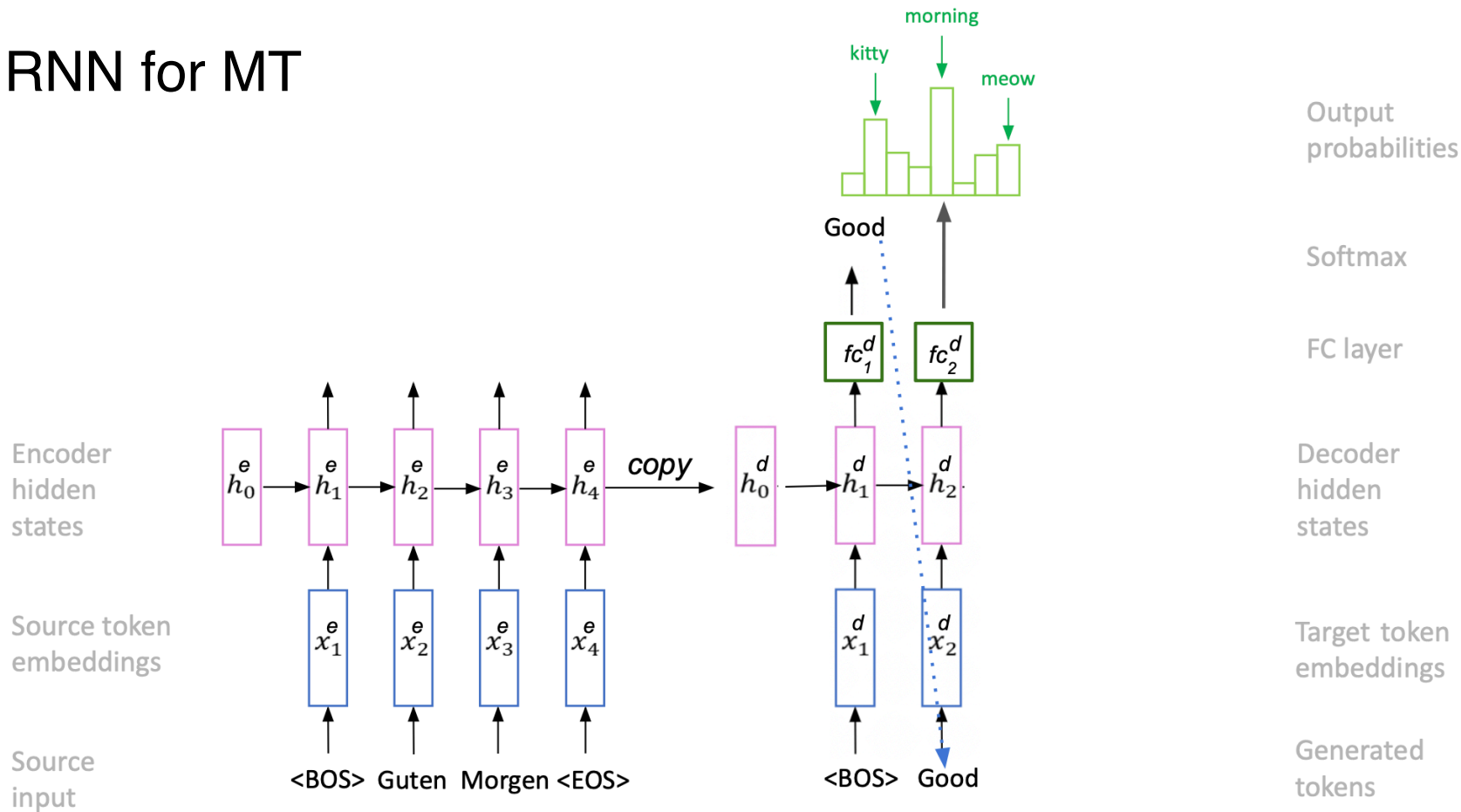
This hidden state contains the information about the input sentence



RNN for MT



RNN for MT

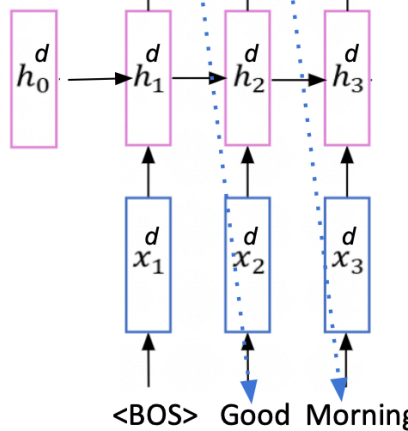
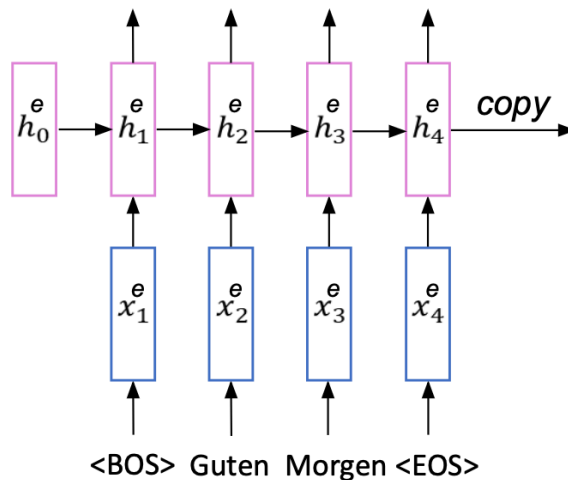


RNN for MT

Encoder
hidden
states

Source token
embeddings

Source
input



Output
probabilities

Softmax

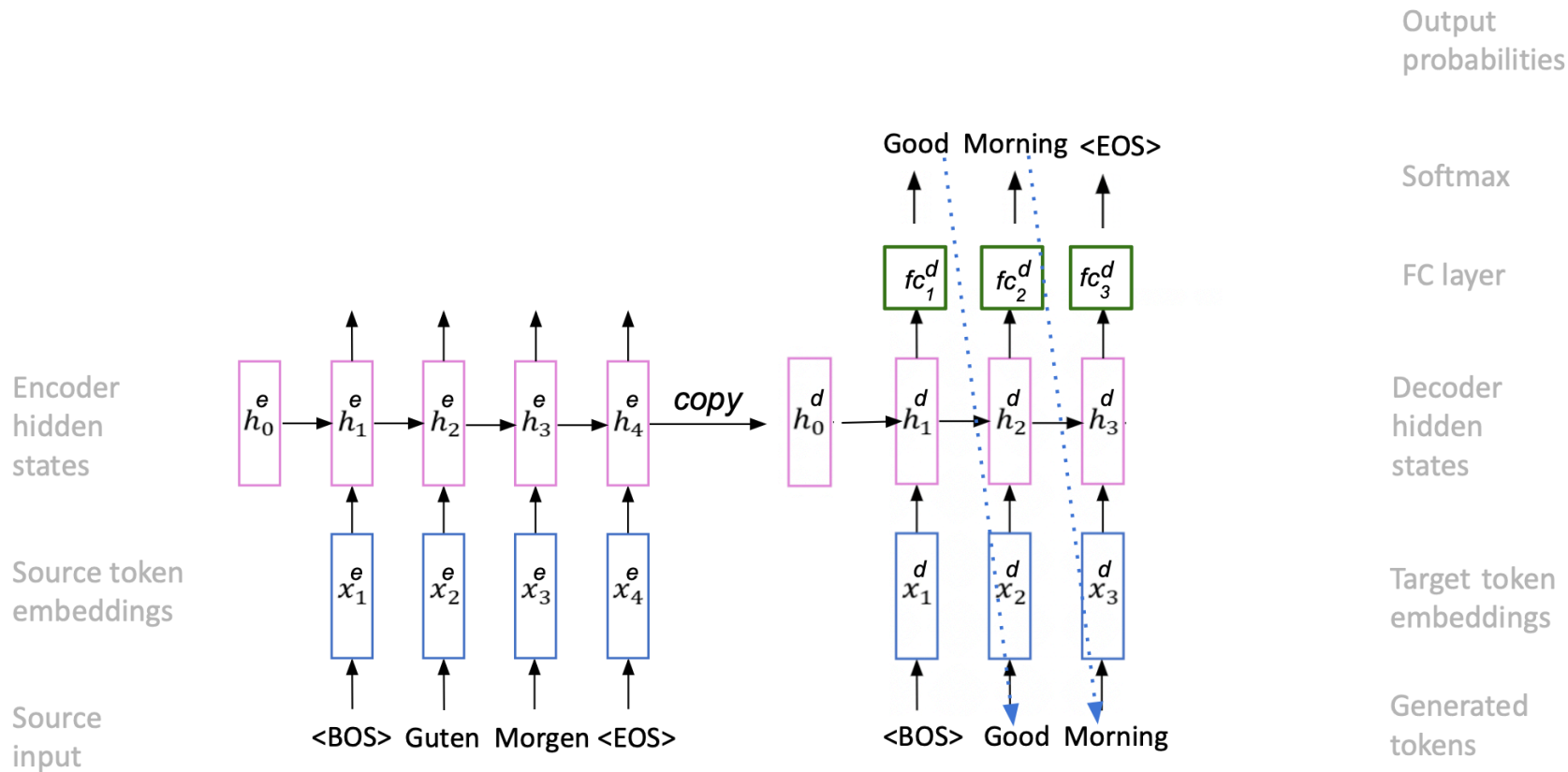
FC layer

Decoder
hidden
states

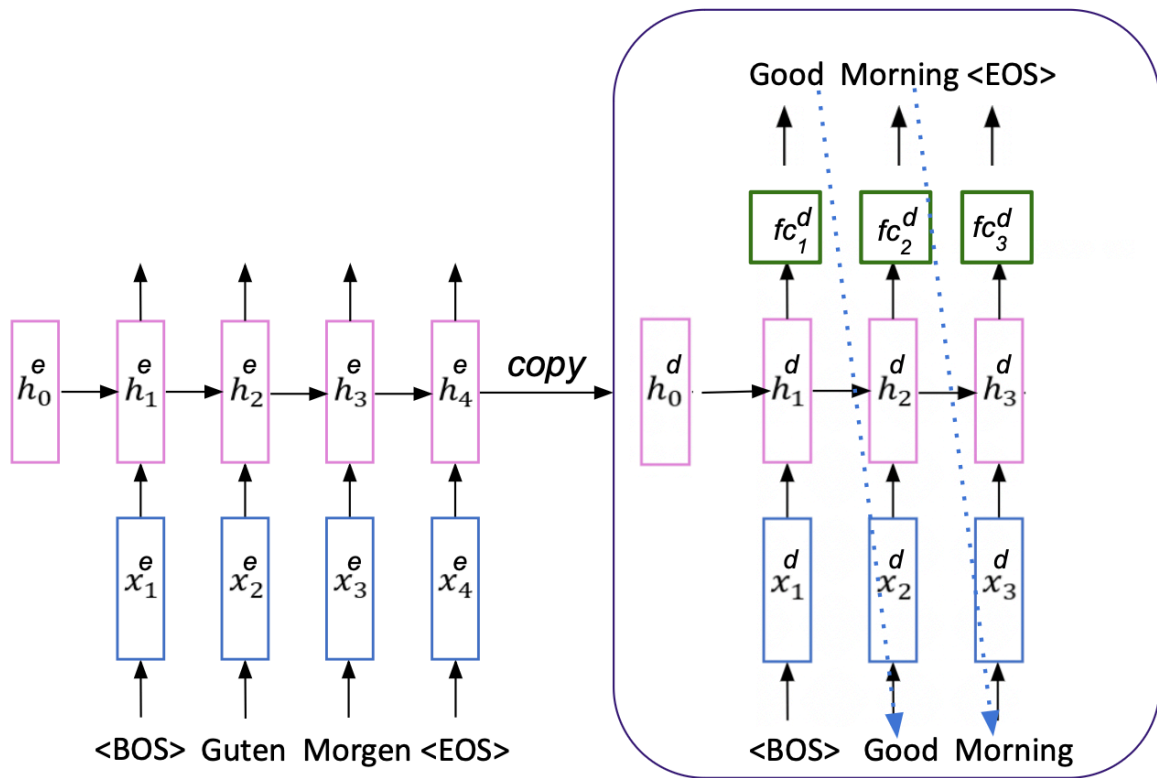
Target token
embeddings

Generated
tokens

RNN for MT

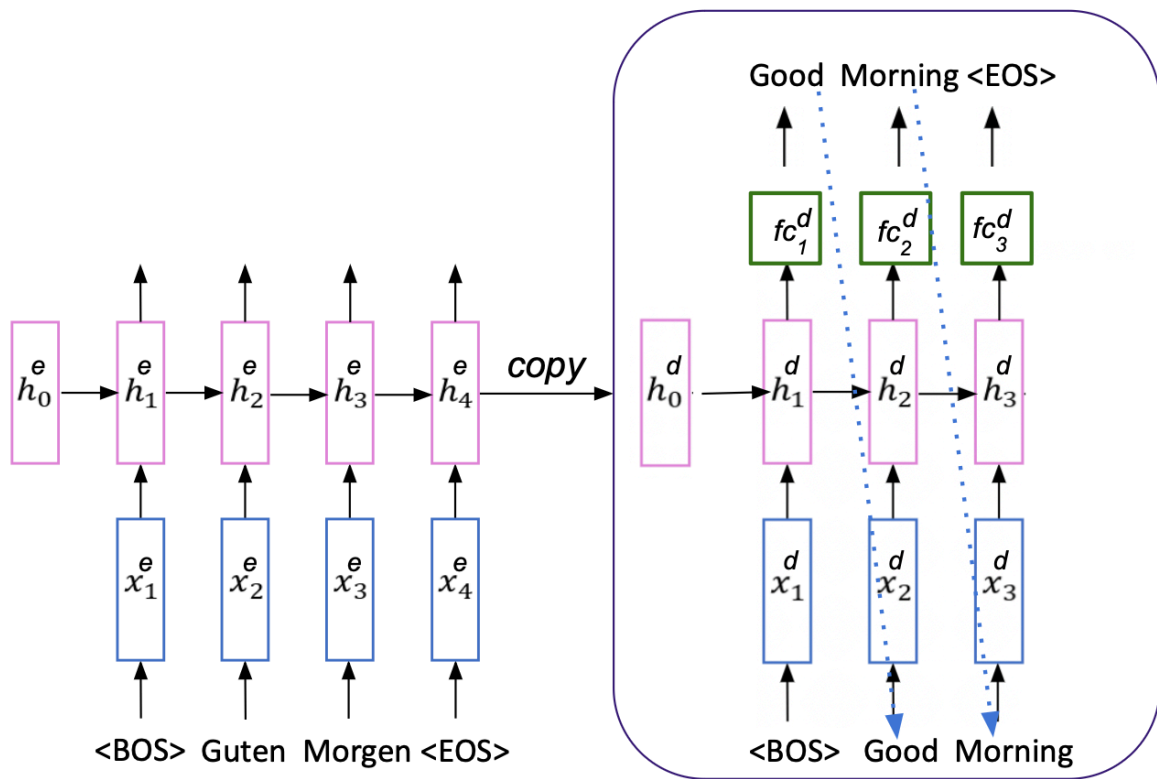


RNN for MT



Conditional
Language
Model

RNN for MT



Such model architecture
is called
Sequence-to-Sequence
(**Seq2Seq**)

Training RNN for MT

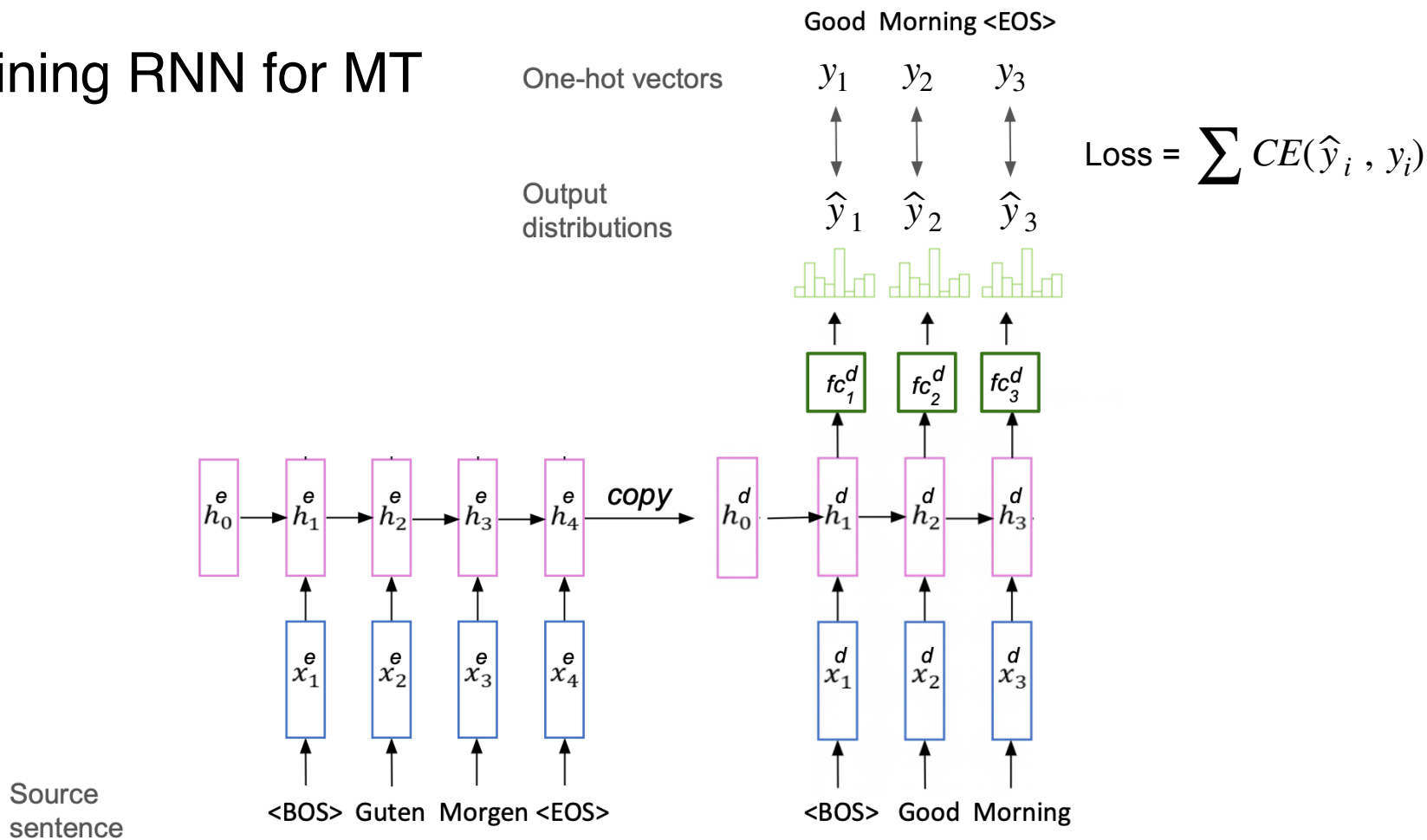
We need training data to train our model

Training data is a corpus of parallel sentences

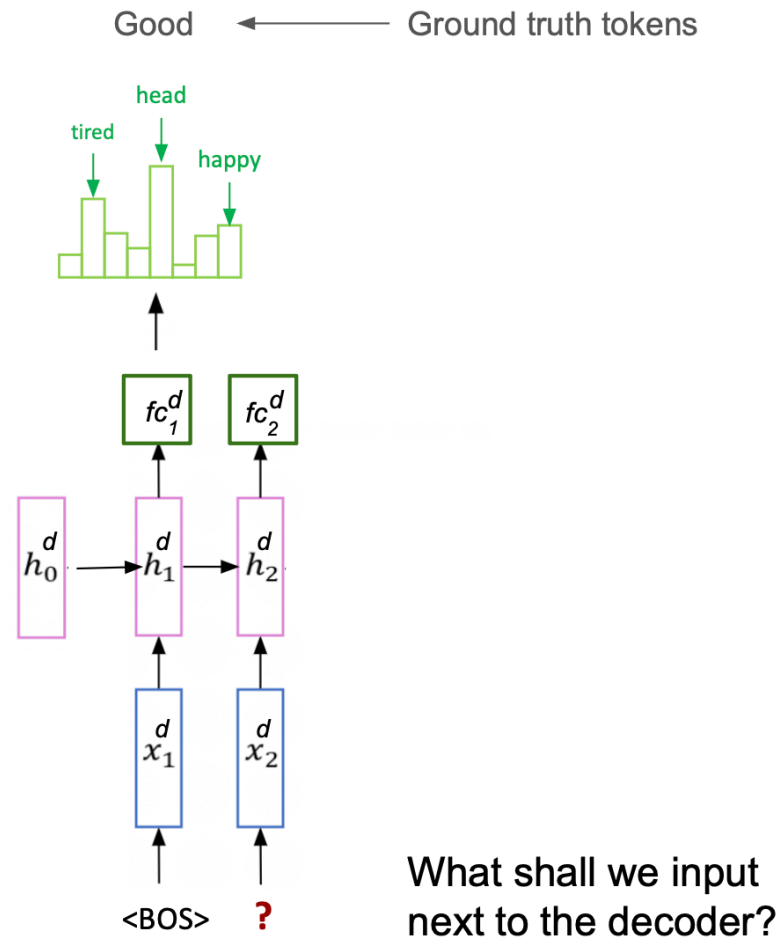
(source sentence, target sentence)

(Guten Morgen, Good Morning)

Training RNN for MT

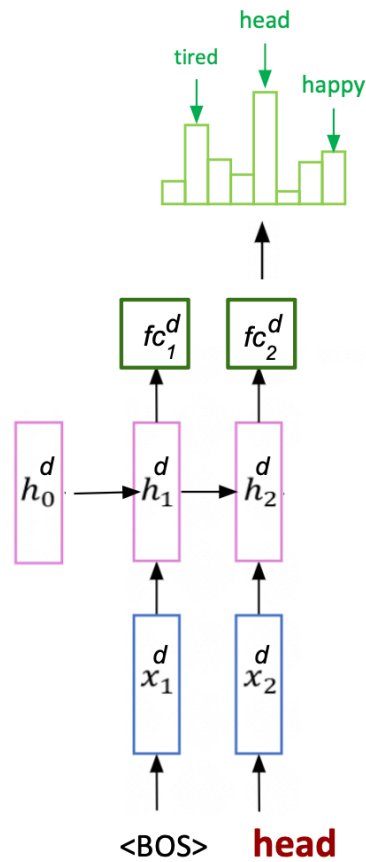


Training RNN for MT



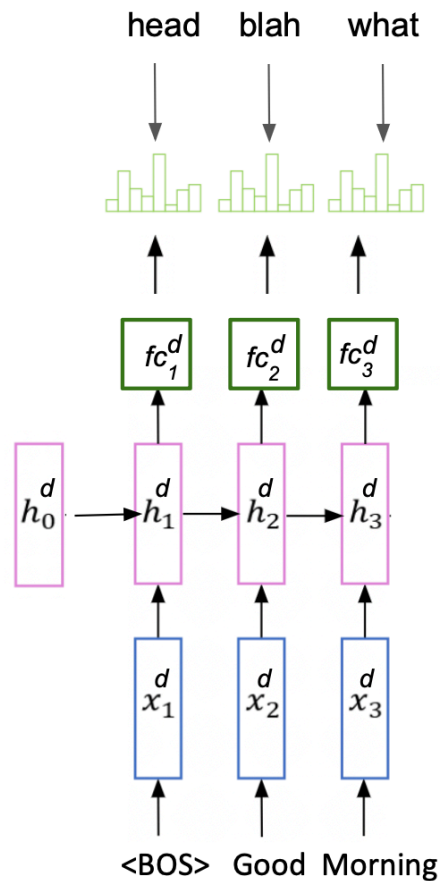
Training RNN for MT

Good morning ← Ground truth tokens



What shall we input next to the decoder?

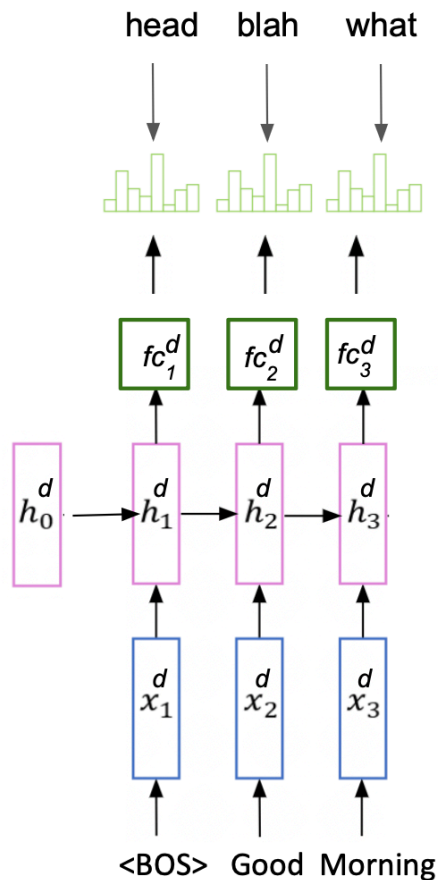
Training RNN for MT



Teacher forcing:

Use target sentence as
input to the decoder
during training

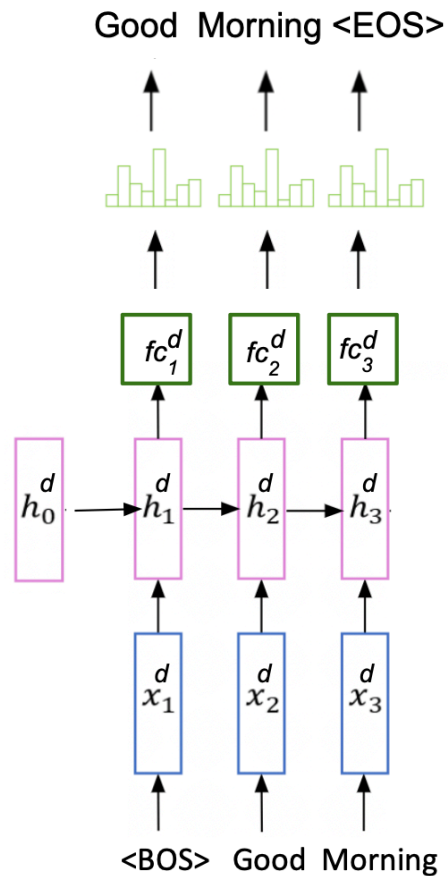
Training RNN for MT



Teacher forcing:

We can use teacher forcing with probability ρ on each iteration, starting from $\rho = 1$ and making it smaller as training progresses.

Training RNN for MT



Teacher forcing:

We cannot use teacher forcing during inference

Training RNN for MT

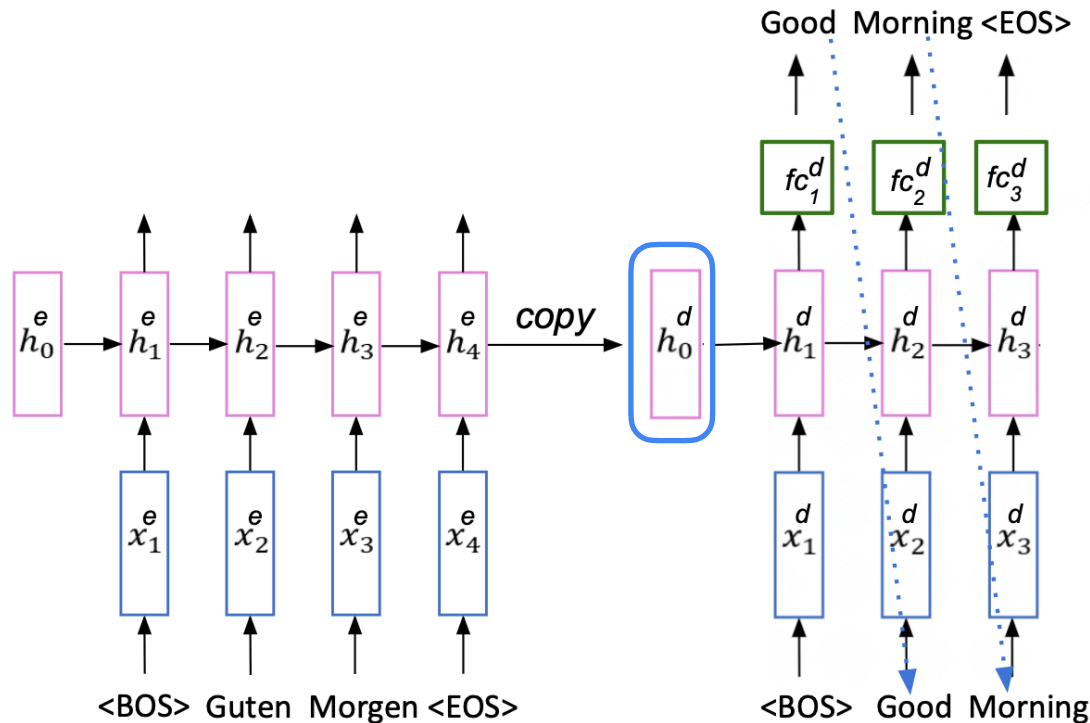
Training tips:

- Encoder and Decoder have separate vocabularies;
- Use subword tokenisation for both source and target vocabs;
- Use beam search when decoding

Training RNN for MT: Problems

Let us now discuss limitations of proposed approach

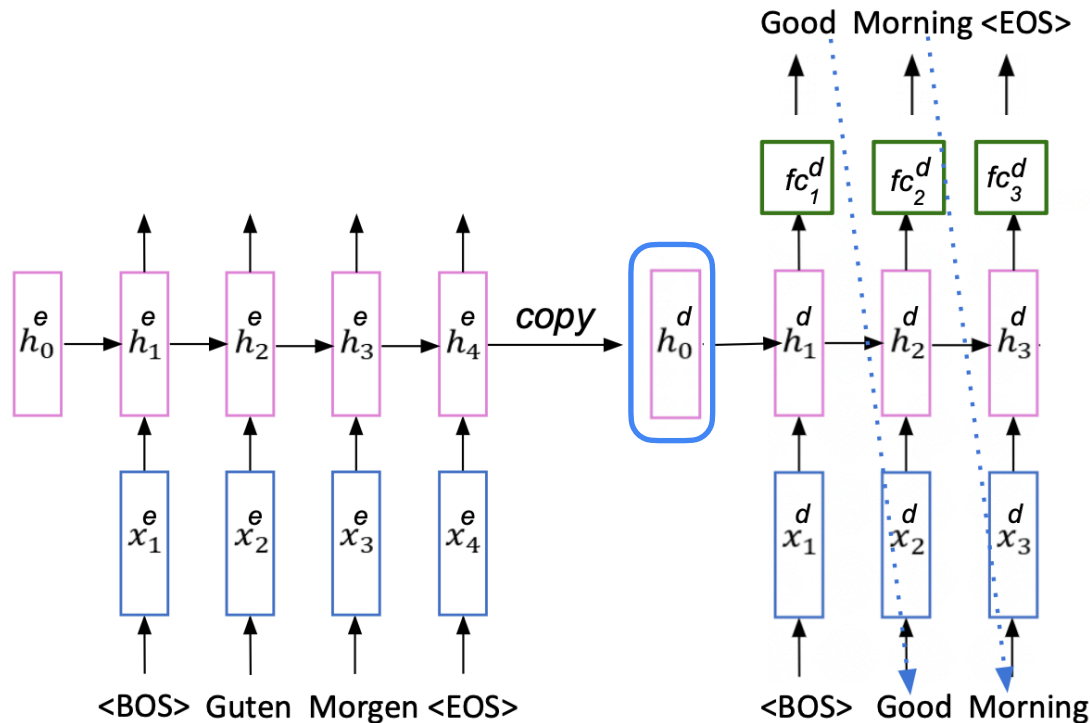
Training RNN for MT: Problems



Limitations:

- Encoder RNN can forget information from the beginning of the input sentence
- One vector h_d^0 is responsible for storing all the information from input sentence. It has limited capacity.

Training RNN for MT: Problems



Ways to overcome these problems:

- Use GRU/LSTM cells
- Use bidirectional RNN

But this does not solve the problem entirely.

Attention Mechanism

Attention: Idea

Imagine we need to translate this sentence from English to German:

*A tall handsome man in a hat with a beautiful parrot on his shoulder
has entered the house on the corner of West India street.*

Attention: Idea

Imagine we need to translate this sentence from English to German:

*A tall handsome man in a hat with a beautiful parrot on his shoulder
has entered the house on the corner of West India street.*

Our RNN works in the following way:

- Read the whole sentence once and “memorise it”
- Generate output sequence

Attention: Idea

Imagine we need to translate this sentence from English to German:

A tall handsome man in a hat with a beautiful parrot on his shoulder has entered the house on the corner of West India street.

What we would do:

- While generating translation, we would look at the initial sentence multiple times to pay attention on parts which are important for generating next couple of words.

Attention: Idea

The idea of Attention was proposed in [this paper](#):

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau

Jacobs University Bremen, Germany

KyungHyun Cho Yoshua Bengio*

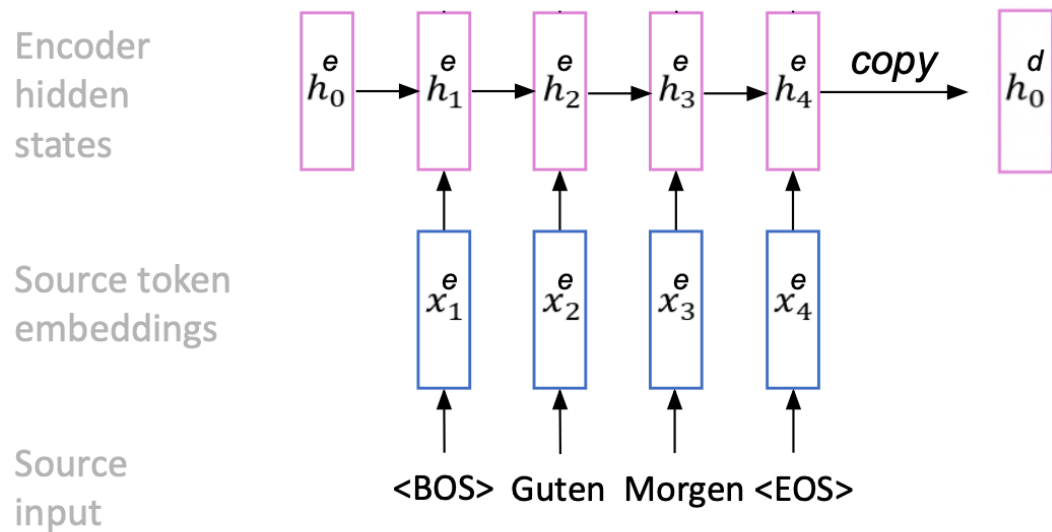
Université de Montréal

ABSTRACT

Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder-decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.

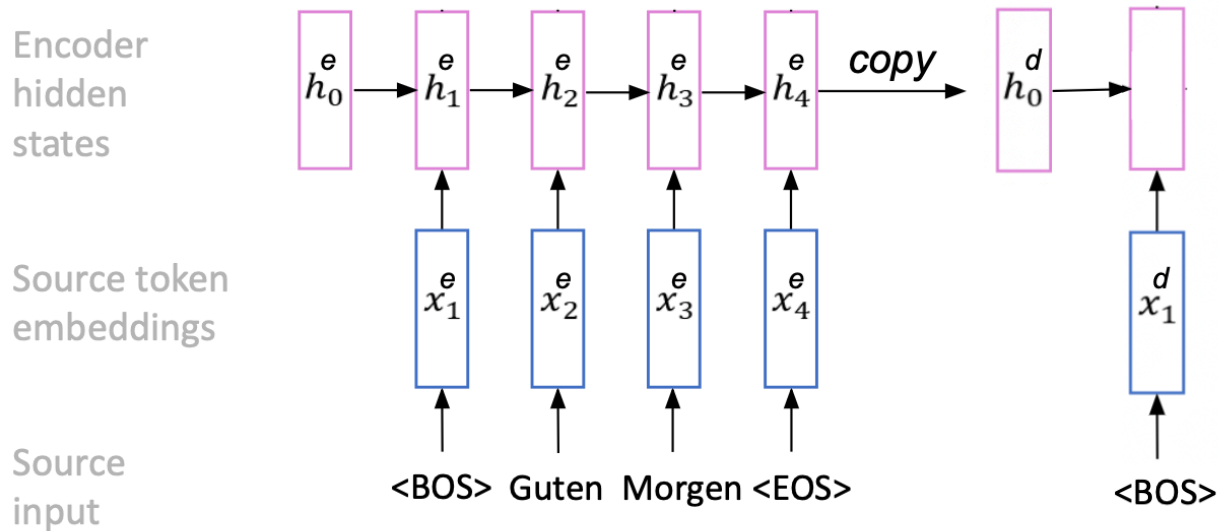
Attention: Idea

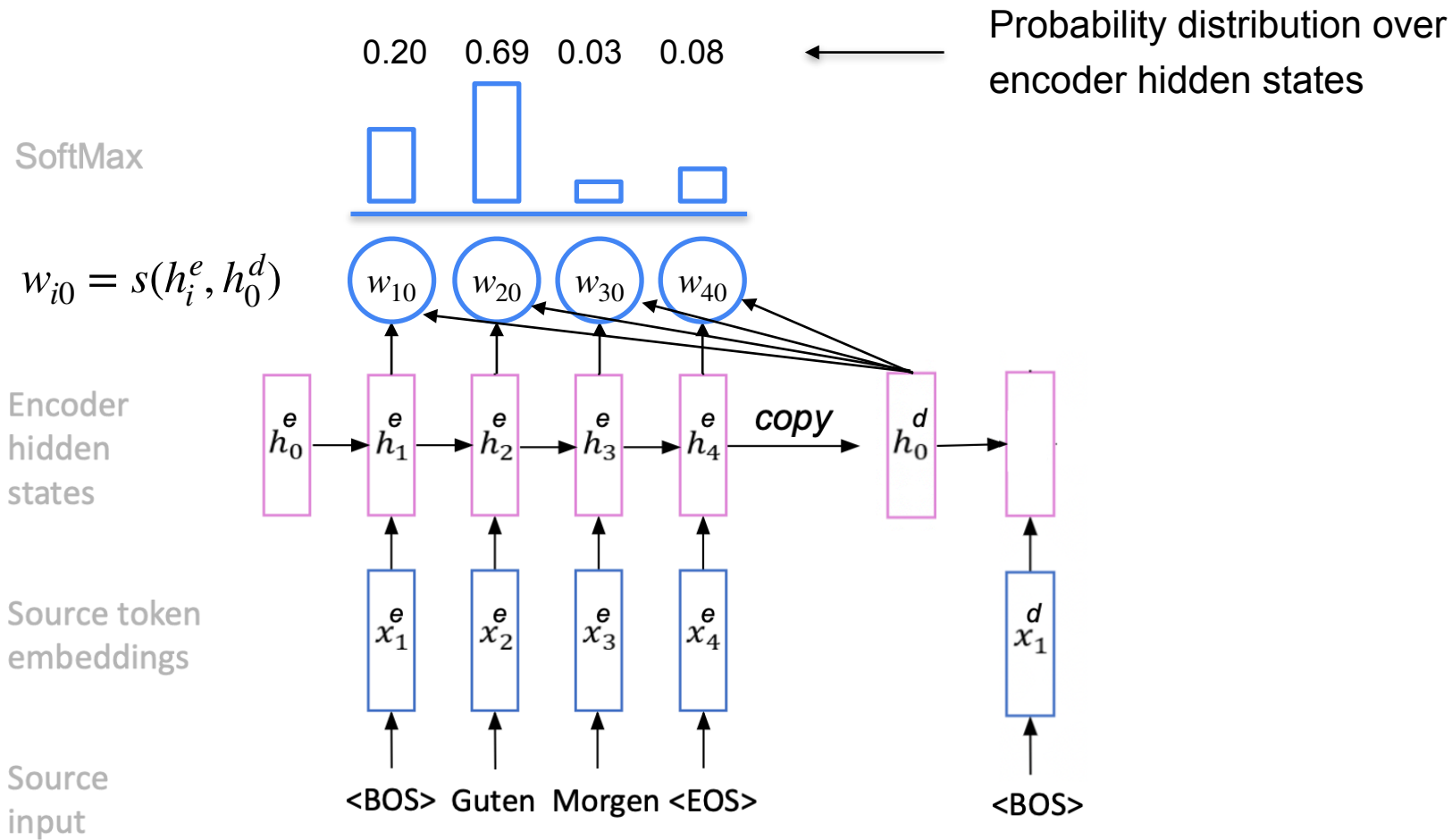
Encoder part works the same way as before



Attention: Idea

Before updating hidden state of the decoder, we need to attend to the input sequence





$$a_0 = \sum h_i^e * w_{i0}$$

Constructing attention vector

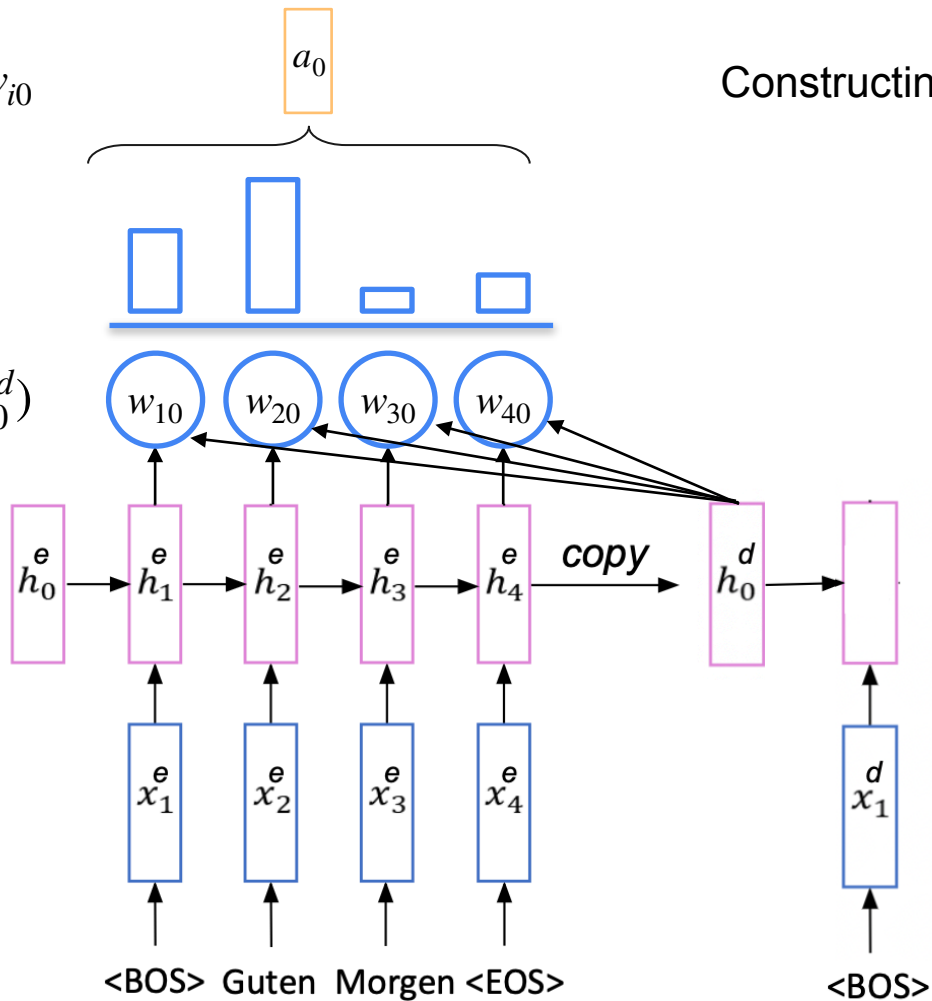
SoftMax

$$w_{i0} = s(h_i^e, h_0^d)$$

Encoder
hidden
states

Source token
embeddings

Source
input



$$a_0 = \sum h_i^e * w_{i0}$$

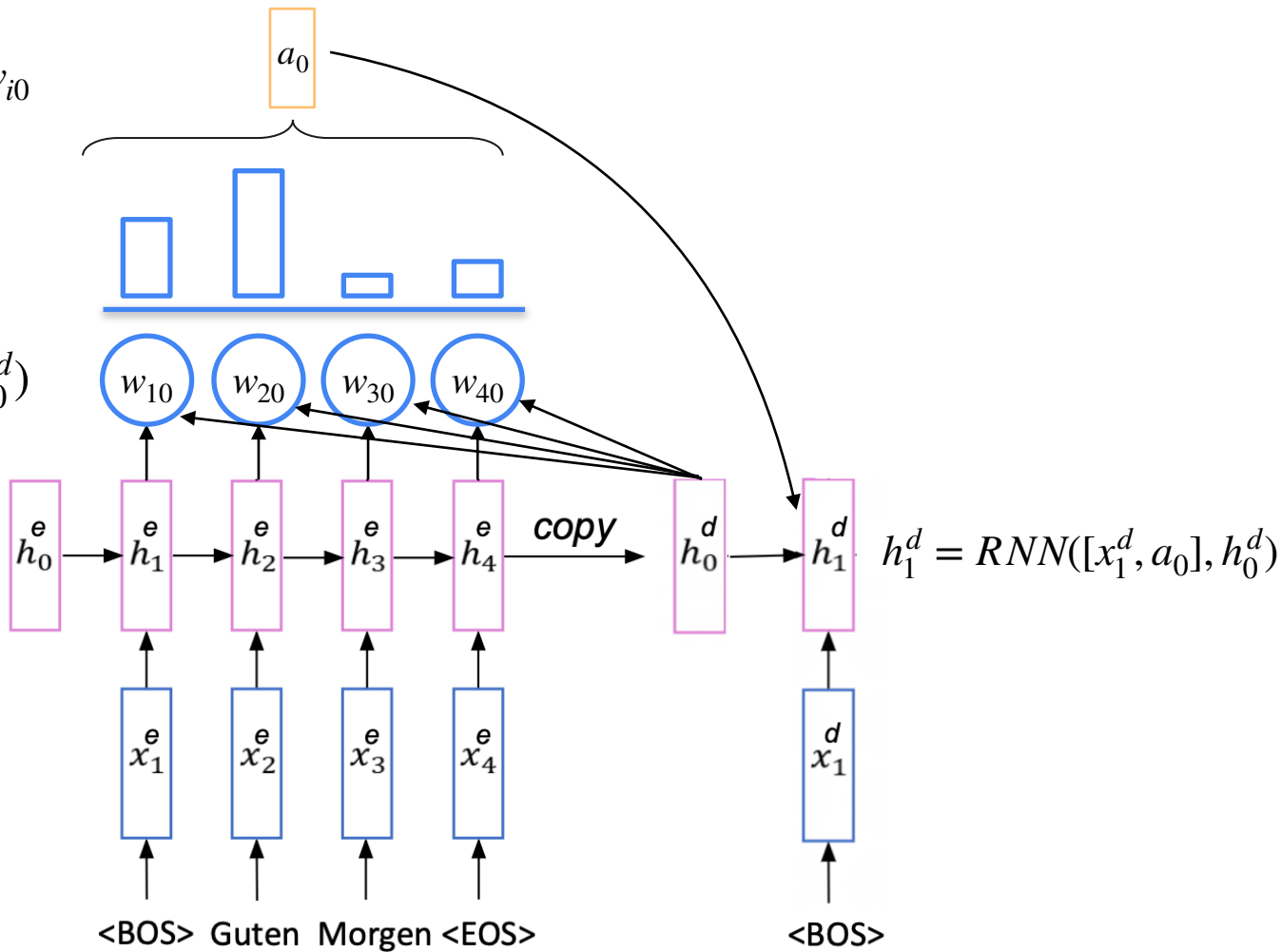
SoftMax

$$w_{i0} = s(h_i^e, h_0^d)$$

Encoder
hidden
states

Source token
embeddings

Source
input



$$a_0 = \sum h_i^e * w_{i0}$$

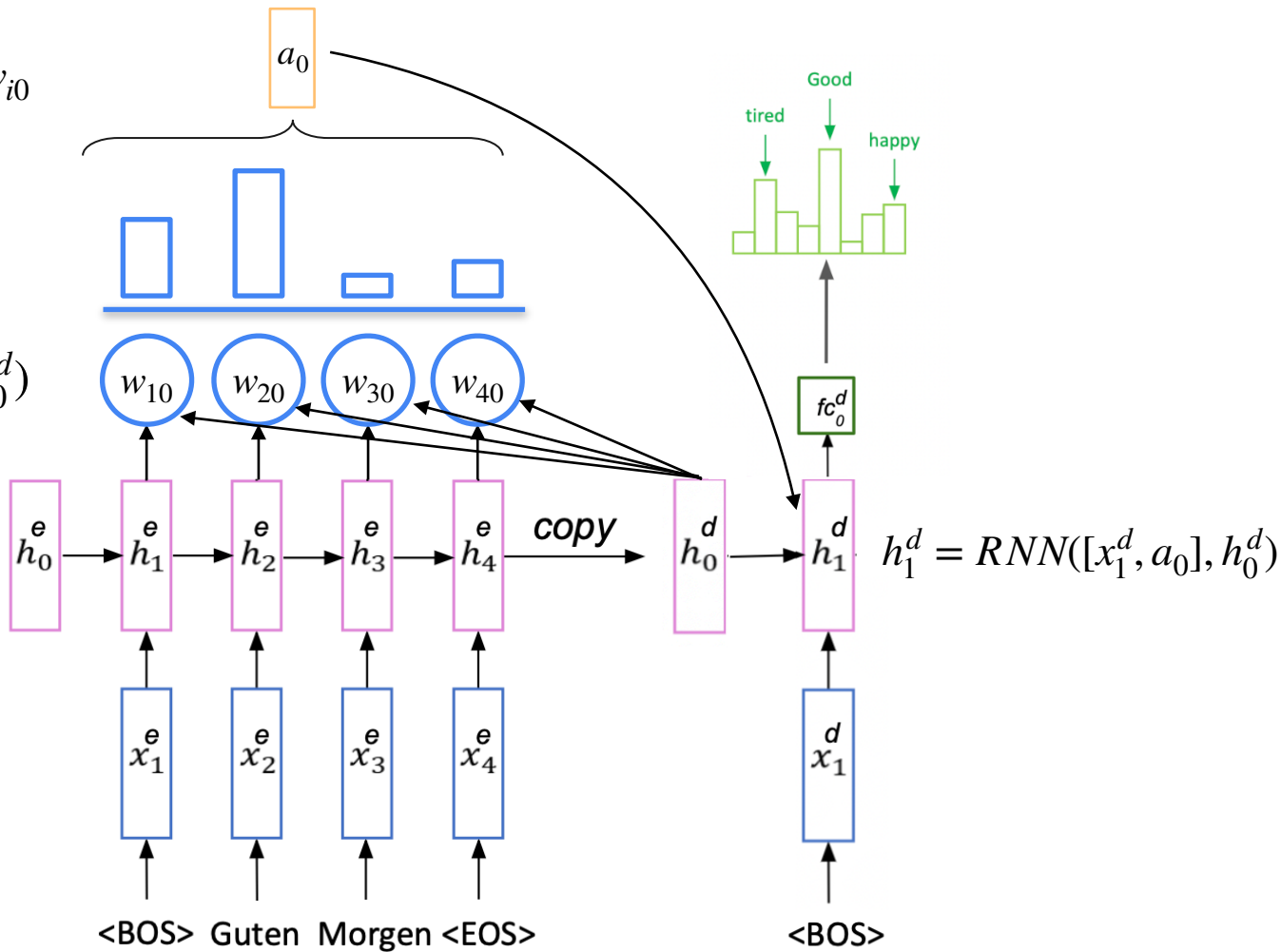
SoftMax

$$w_{i0} = s(h_i^e, h_0^d)$$

Encoder
hidden
states

Source token
embeddings

Source
input



$$a_0 = \sum h_i^e * w_{i1}$$

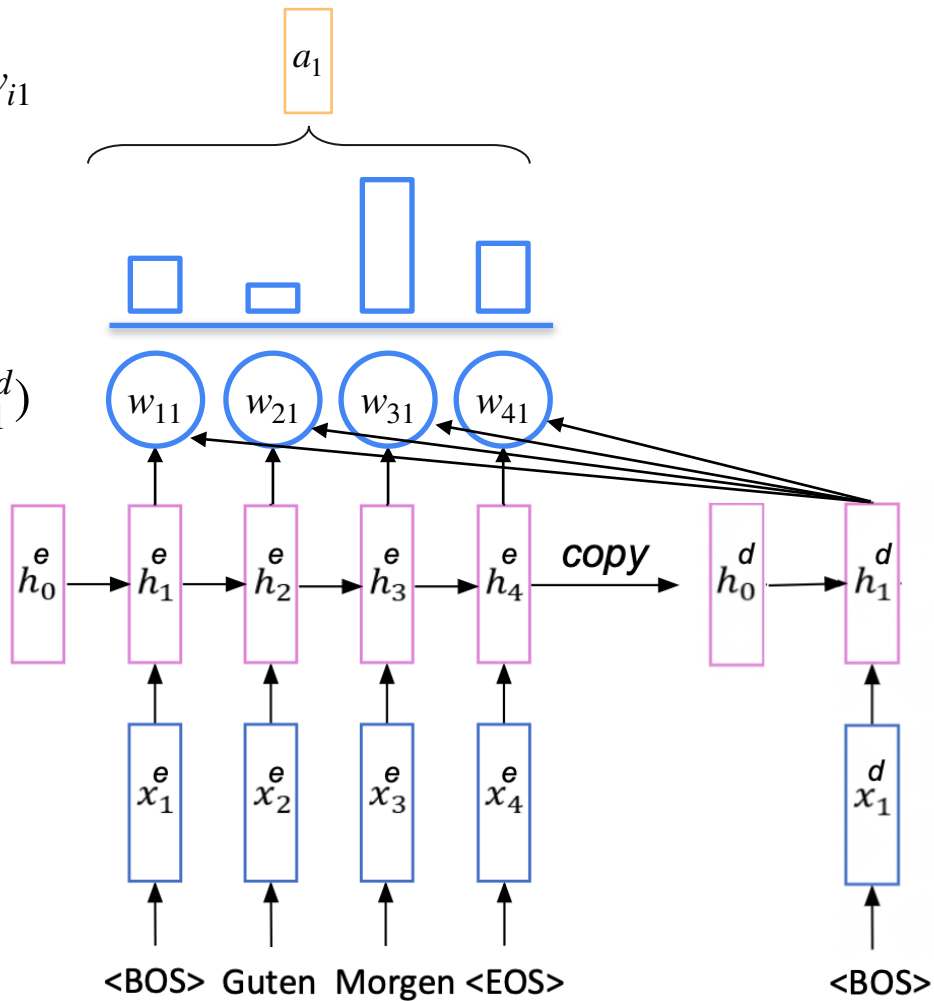
SoftMax

$$w_{i1} = s(h_i^e, h_1^d)$$

Encoder
hidden
states

Source token
embeddings

Source
input



$$a_0 = \sum h_i^e * w_{i1}$$

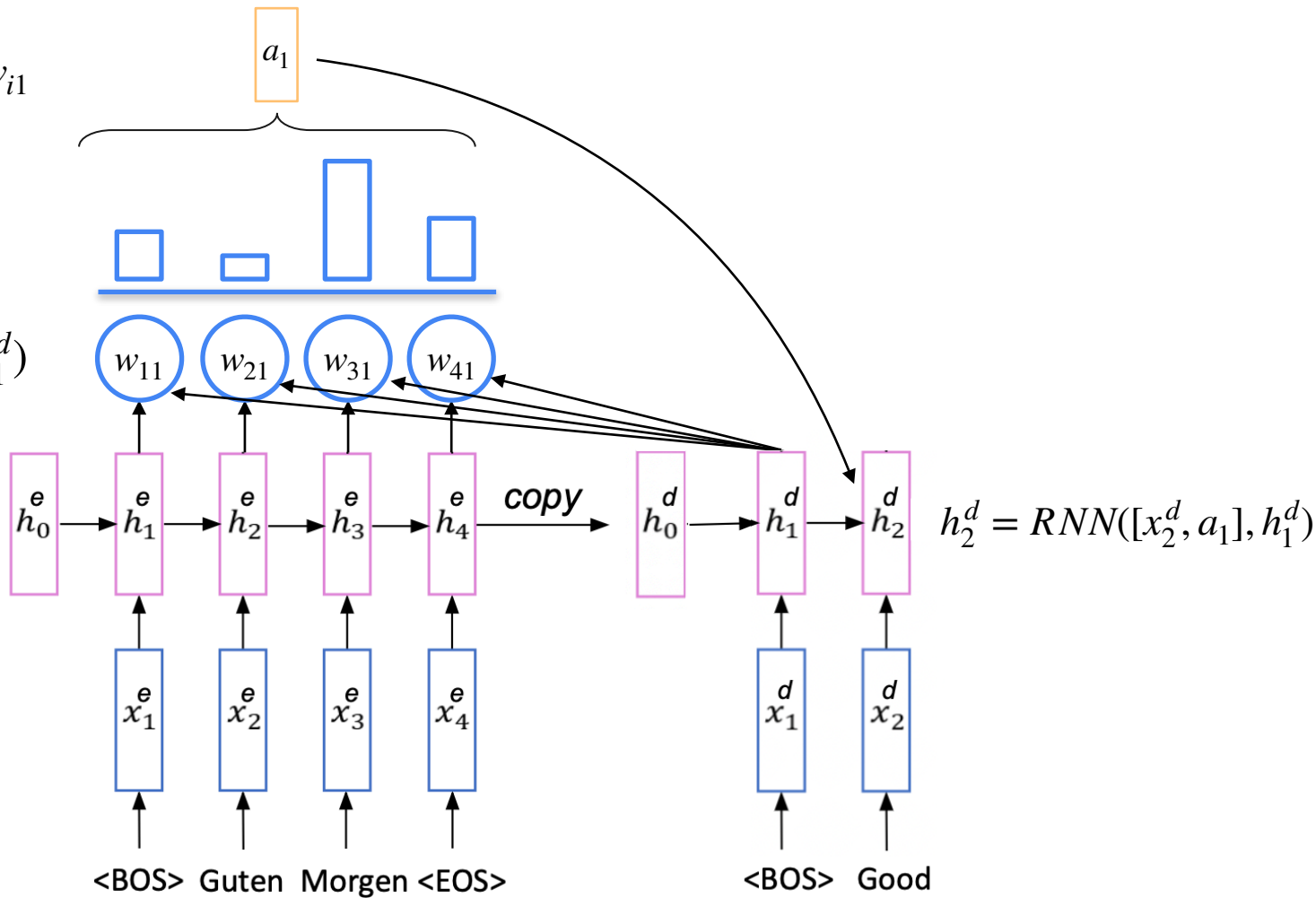
SoftMax

$$w_{i1} = s(h_i^e, h_1^d)$$

Encoder
hidden
states

Source token
embeddings

Source
input



$$a_0 = \sum h_i^e * w_{i1}$$

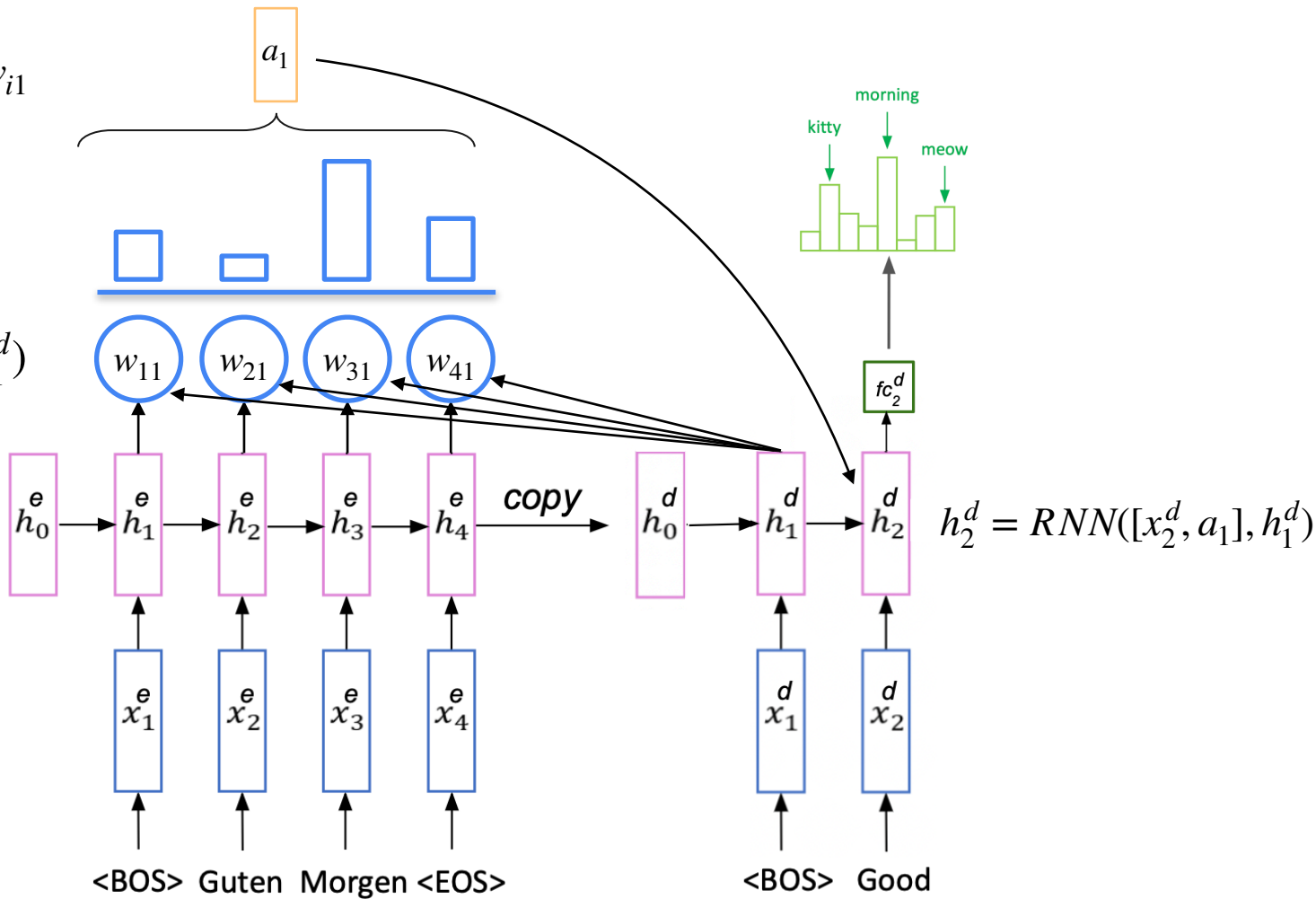
SoftMax

$$w_{i1} = s(h_i^e, h_1^d)$$

Encoder
hidden
states

Source token
embeddings

Source
input



$$a_0 = \sum h_i^e * w_{i1}$$

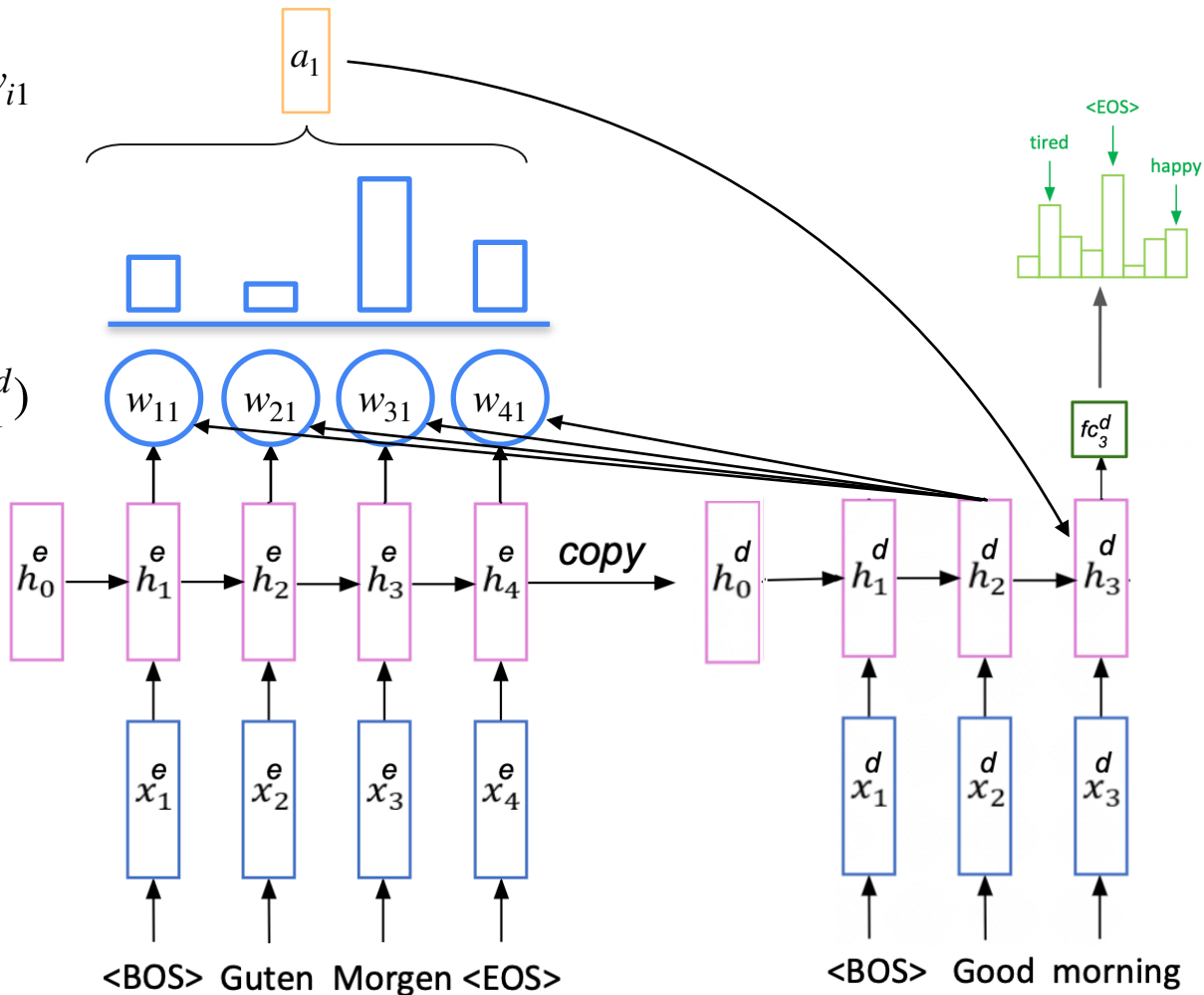
SoftMax

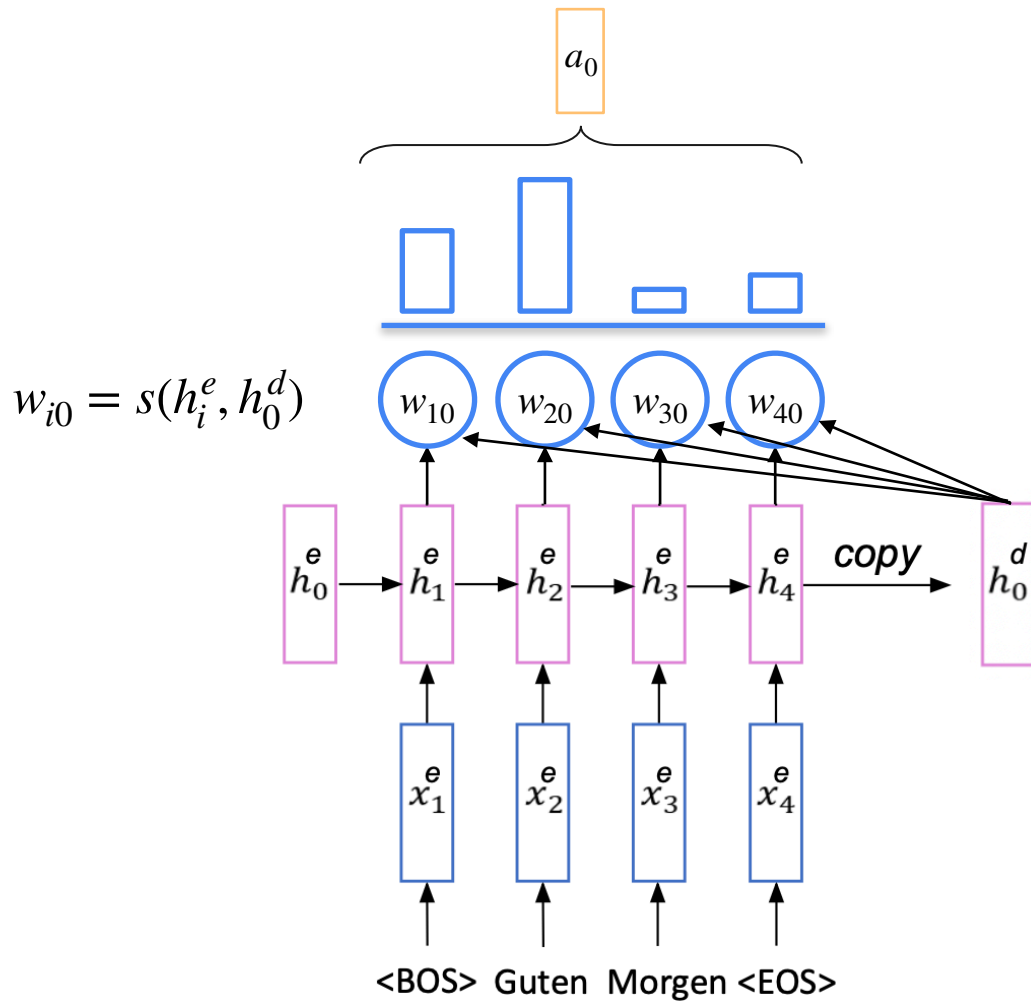
$$w_{i1} = s(h_i^e, h_1^d)$$

Encoder
hidden
states

Source token
embeddings

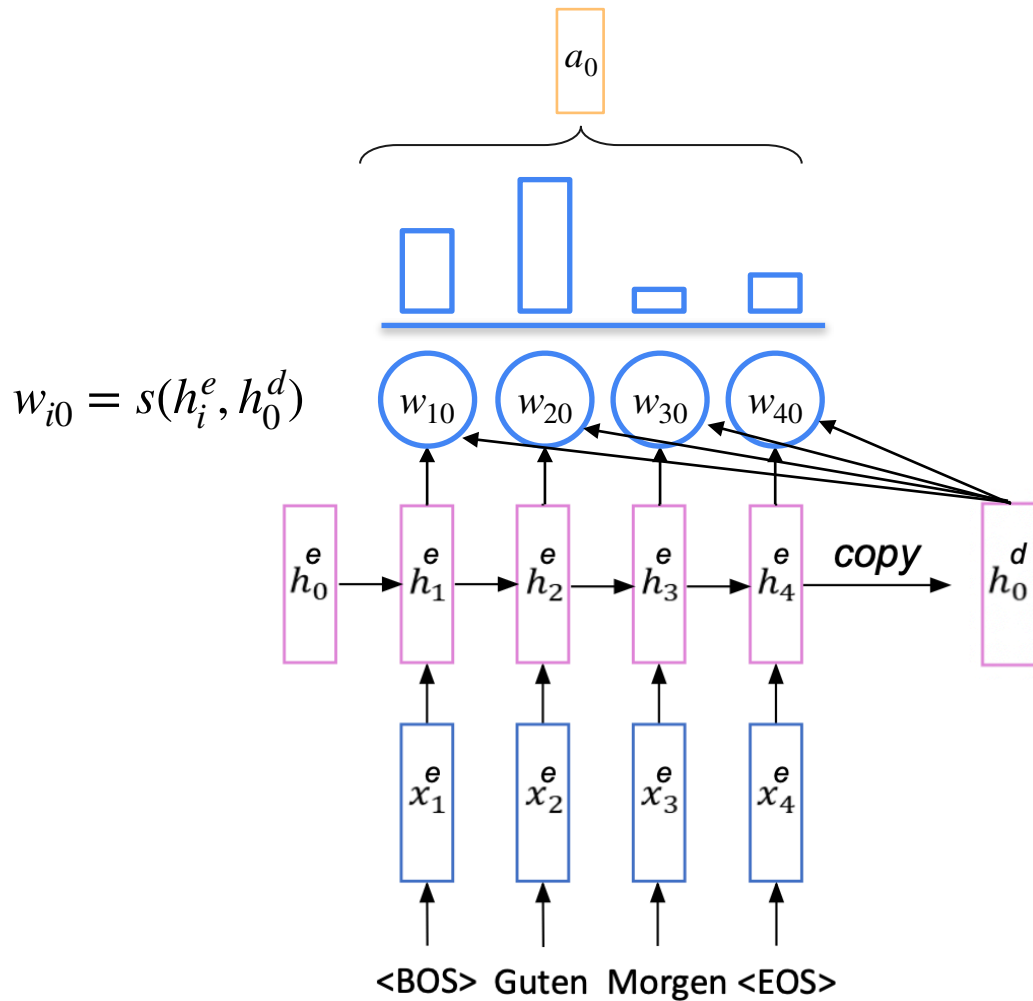
Source
input





Attention function can be trainable or non-trainable

If it is trainable, its weights are trained together with all other network parameters



Versions of attention function:

$$w = s(h_1, h_2)$$

- Dot-product attention (non-trainable)

$$s(h_1, h_2) = h_1^T h_2$$

- One-layer NN (trainable)

$$s(h_1, h_2) = \sigma(W[h_1, h_2])$$

- Many others

Effect of the Attention on MT task metrics

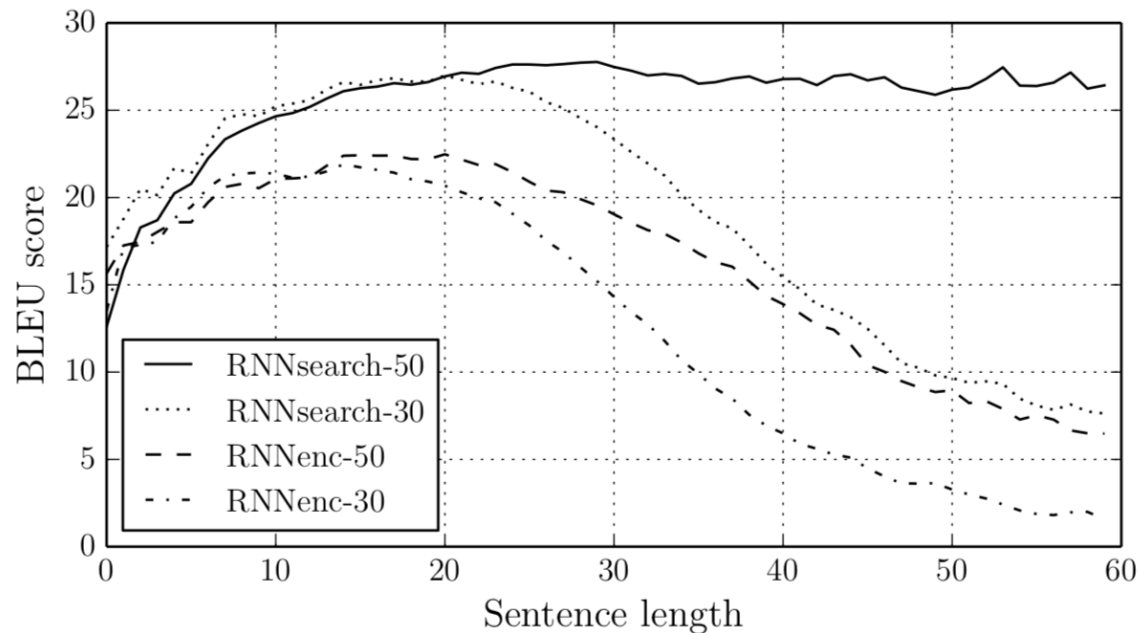
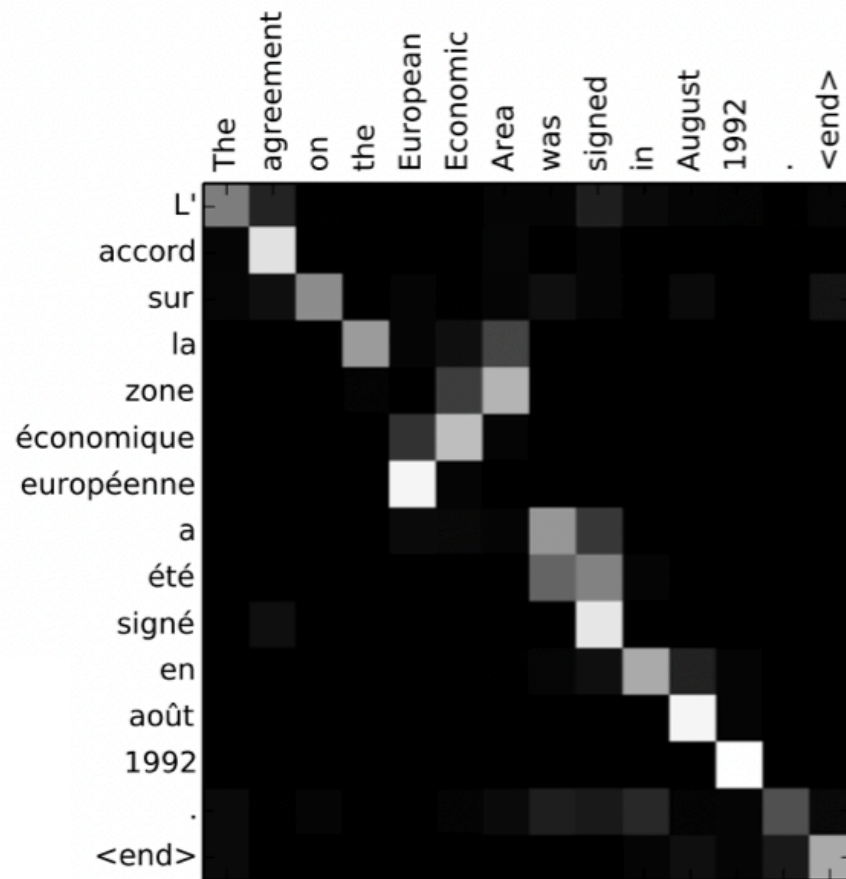
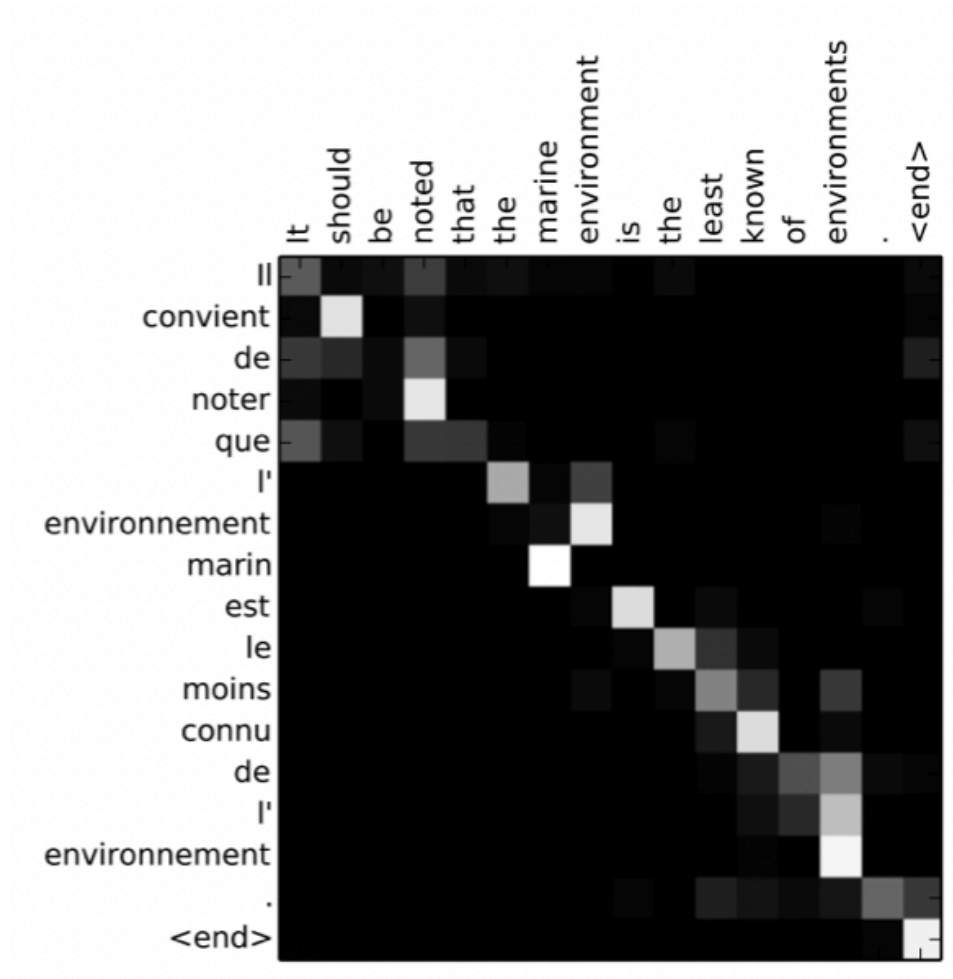


Figure 2: The BLEU scores of the generated translations on the test set with respect to the lengths of the sentences. The results are on the full test set which includes sentences having unknown words to the models.

Attention maps



Attention maps



Attention maps

Inventing Attention mechanism was a pivotal moment in Machine Translation task and all NLP domain.

In 2017, Google presented novel neural network architecture called Transformer, that had Attention as its core mechanism.

Transformers made a revolution in NLP and were further adapted to other domains as well.

Transformer Architecture

Limitations of RNNs

- ***Using RNN in Encoder is slow:***
 - *Forward* process is slow as it processes tokens sequentially
 - *Backward* process is slow as we need to backpropagate gradients through time
- ***RNNs suffer from training problems:***
 - Exploding/vanishing gradients
 - Bad GPU utilisation

Idea of Transformer

Do we actually still need RNNs?

RNNs were chosen for MT task as they can gather information from input sentence and pass it to the decoder.

But we now have Attention mechanism which allows us to “look” at parts of inputs sentence at any decoder step.

Idea of Transformer

Do we actually still need RNNs?

RNNs were chosen for MT task as they can gather information from input sentence and pass it to the decoder.

But we now have Attention mechanism which allows us to “look” at parts of inputs sentence at any decoder step.

Transformer is a neural network architecture which is based on Attention mechanism.

It does not have any RNN layers.

Idea of Transformer

Transformer architecture was first presented in 2017 in a paper by Google called “***Attention is all you need***”

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

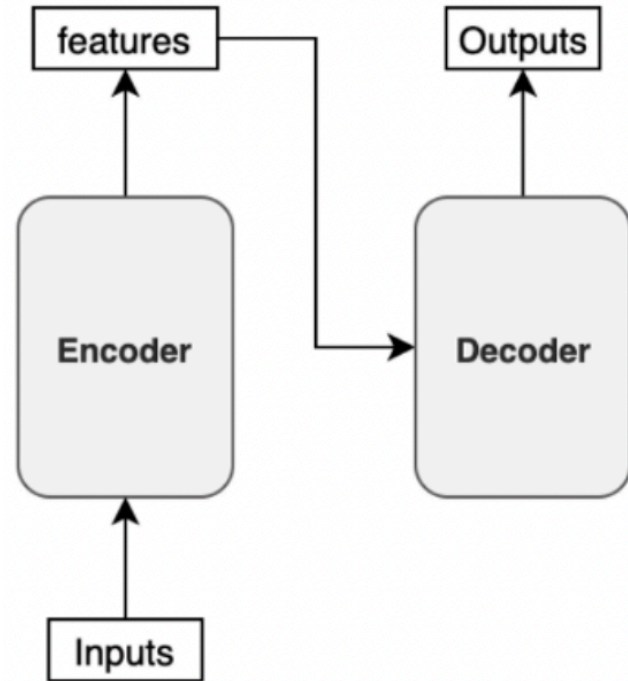
Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* †
illia.polosukhin@gmail.com

Abstract

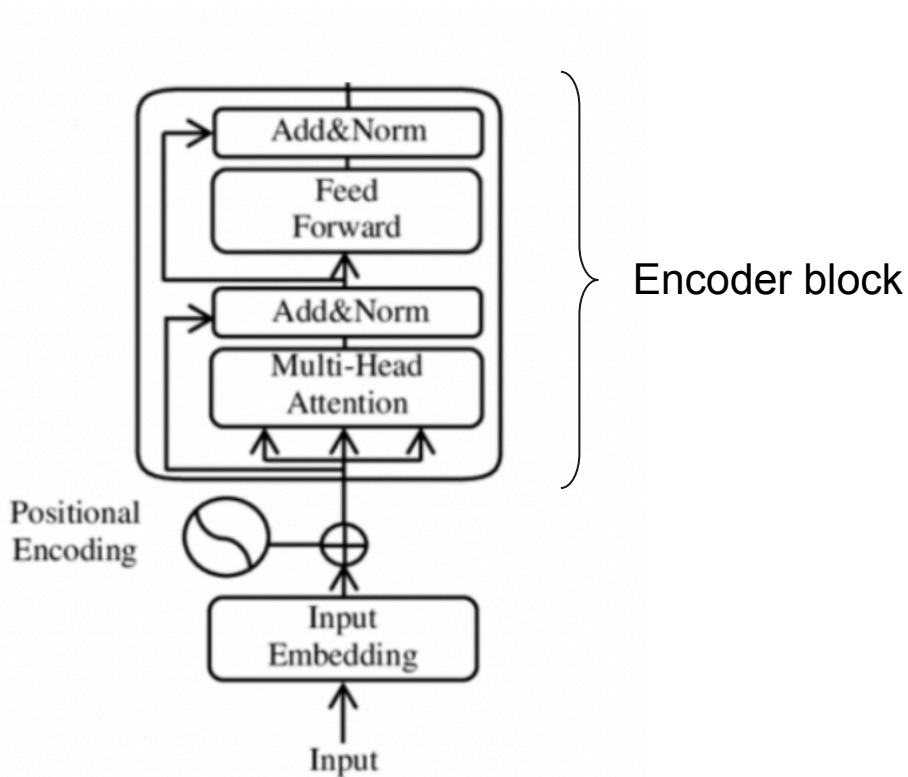
Idea of Transformer

Transformer also has
encoder-decoder structure

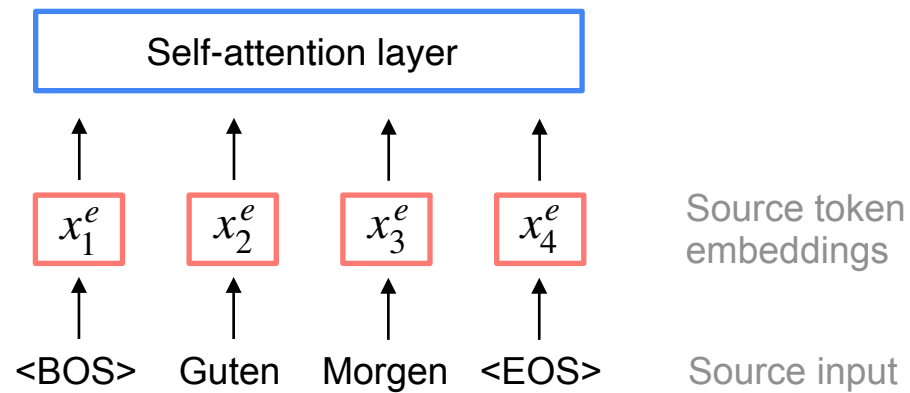


Idea of Transformer

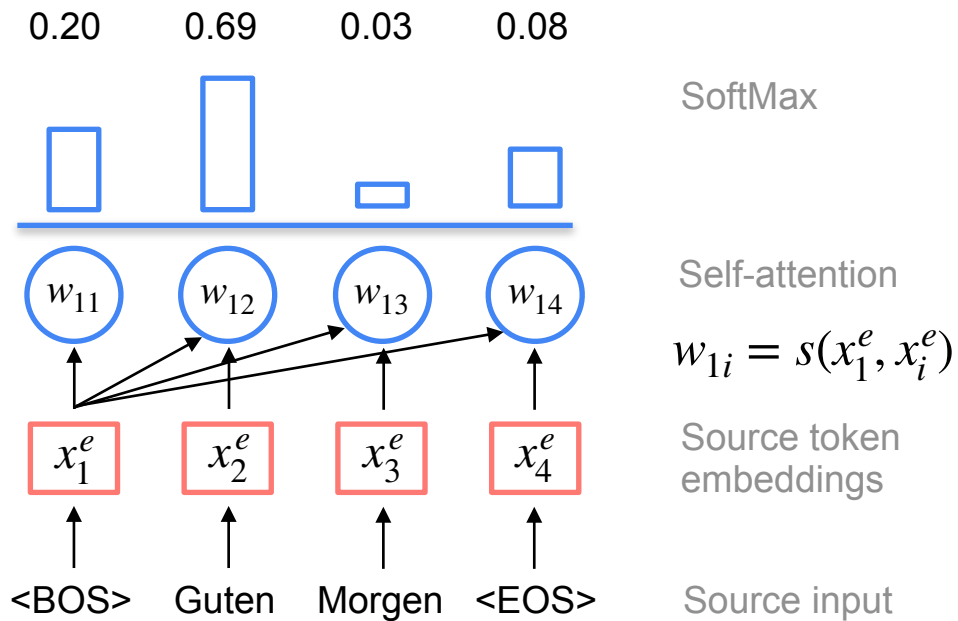
Encoder processes the input sequence *in parallel*



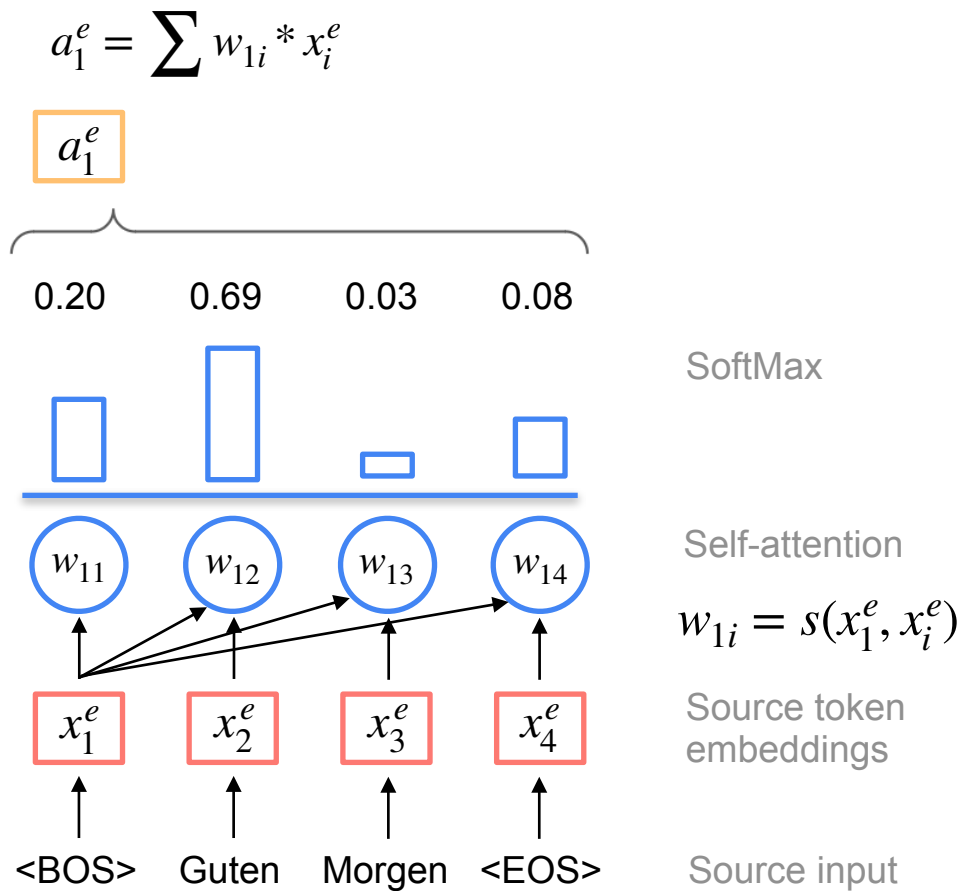
Transformer Encoder



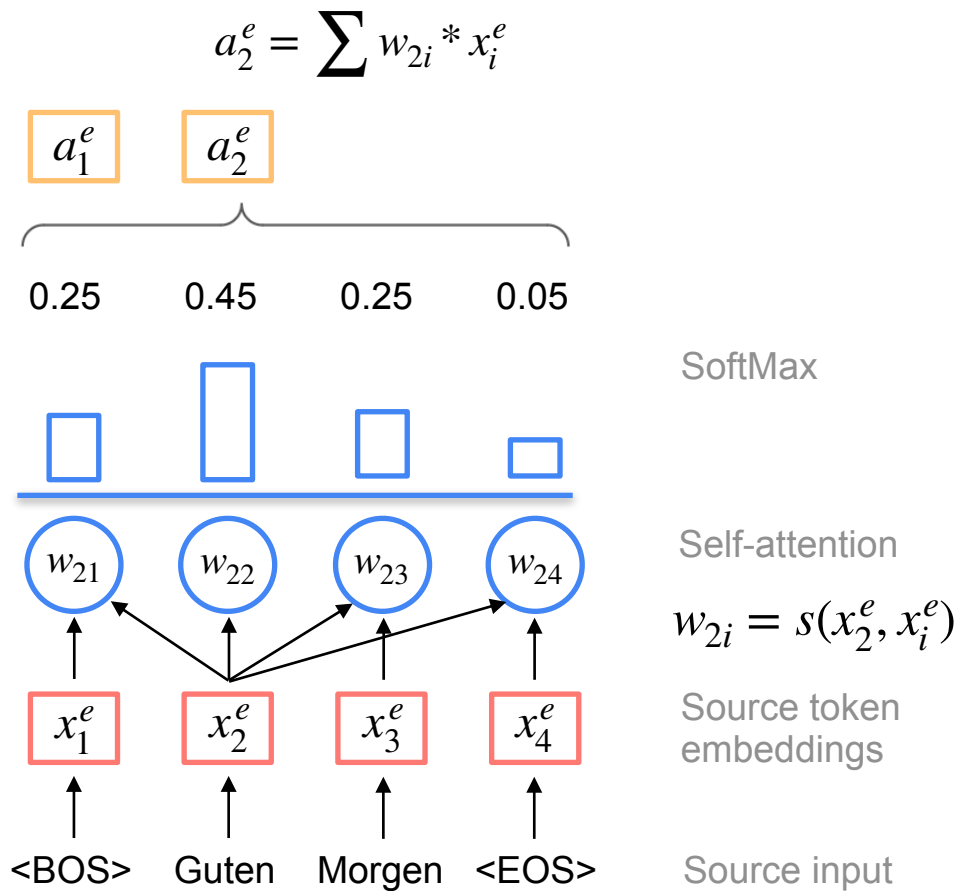
Transformer Encoder



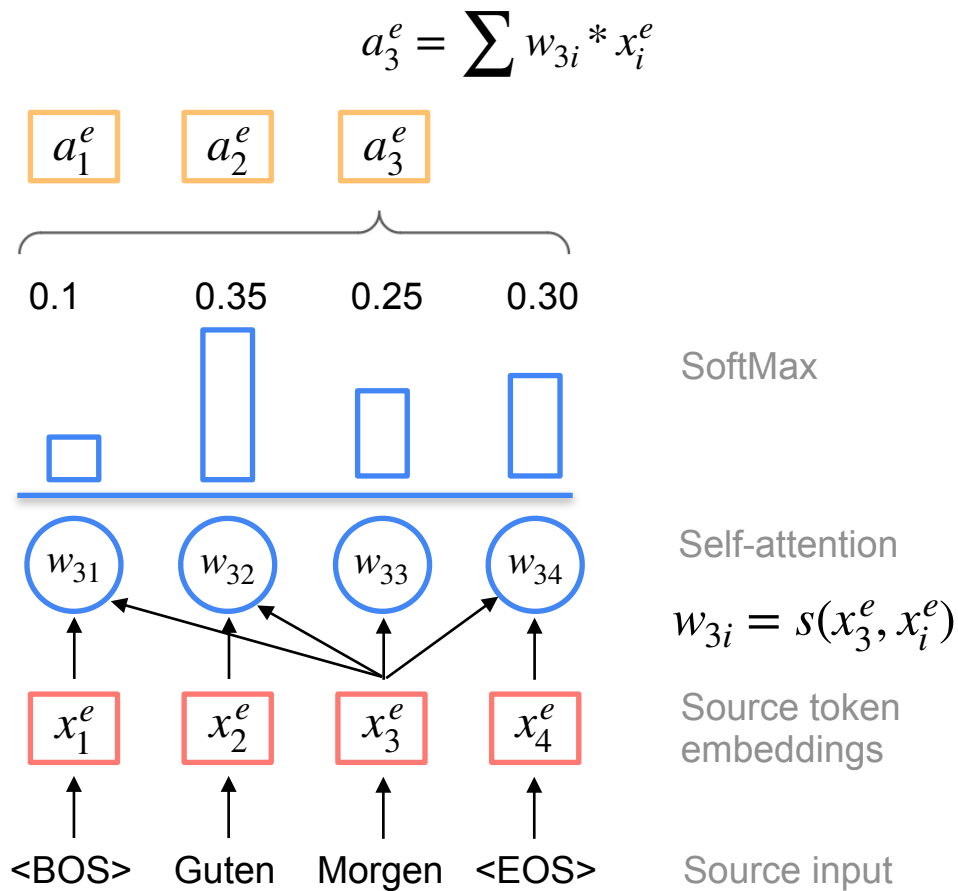
Transformer Encoder



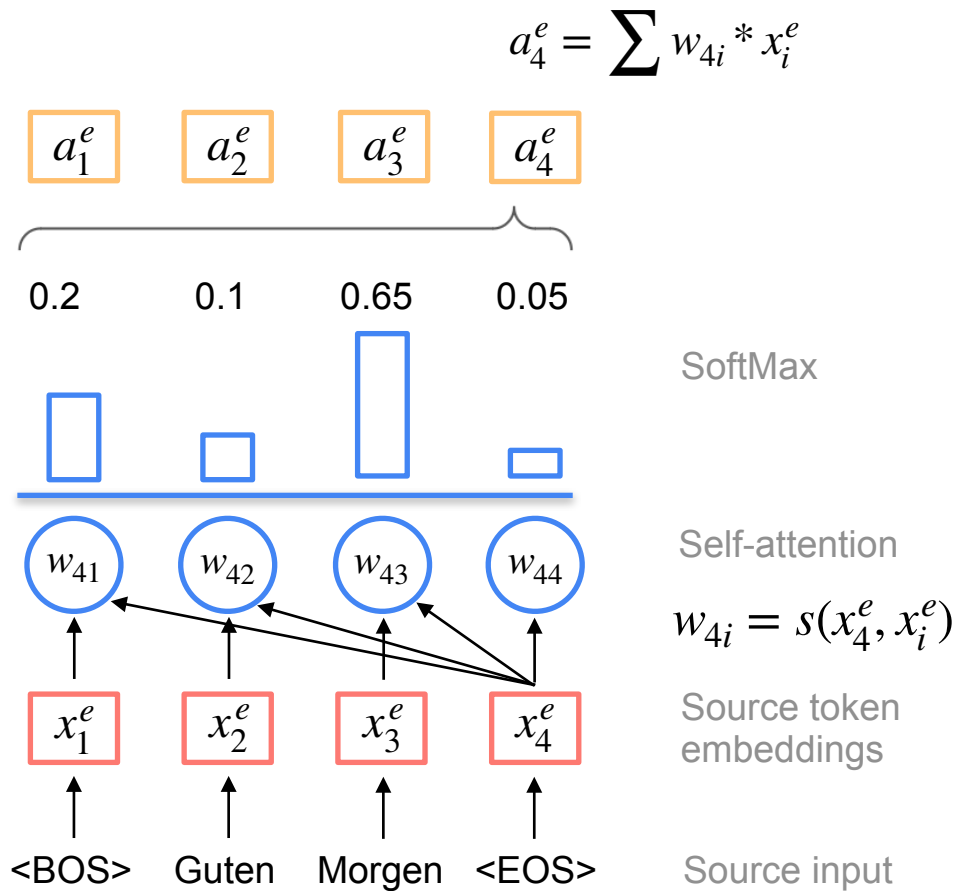
Transformer Encoder



Transformer Encoder

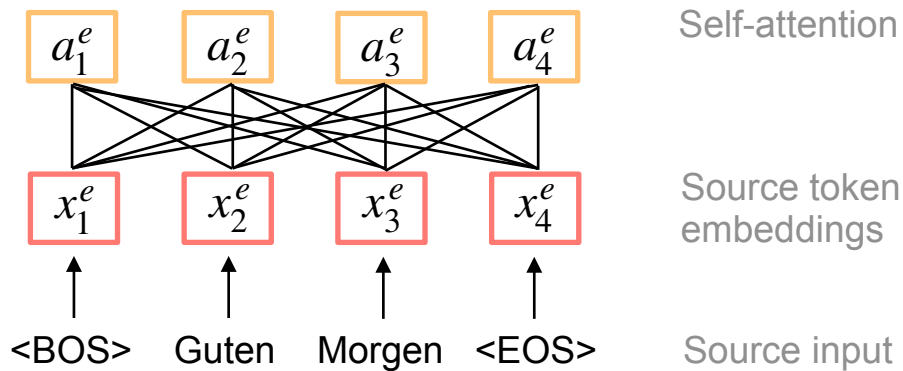


Transformer Encoder



Transformer Encoder

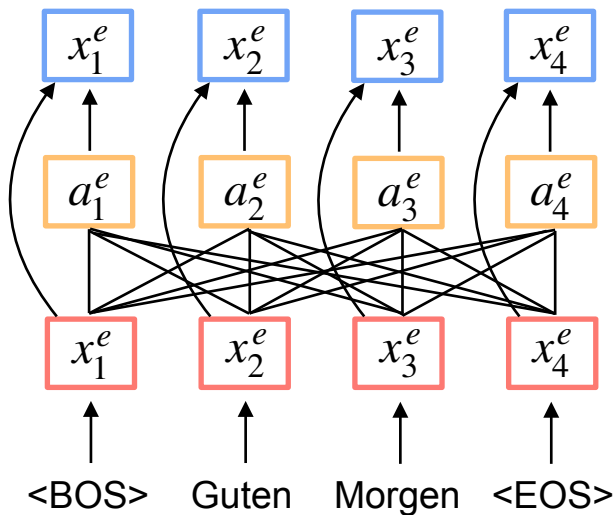
Self-attention is applied to all the token embeddings in parallel



Transformer Encoder

Self-attention layer calculates additional information for each token embedding based on its context (all other token embeddings)

“Sport is good, it helps a lot”



Add
 $x_i^e = x_i^e + a_i^e$

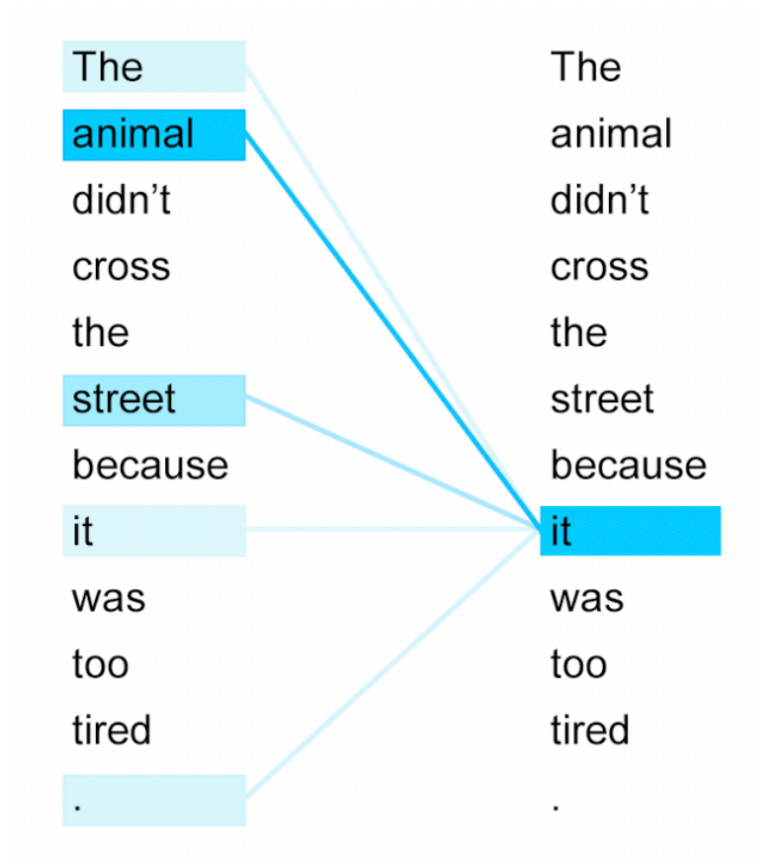
Self-attention

Source token
embeddings

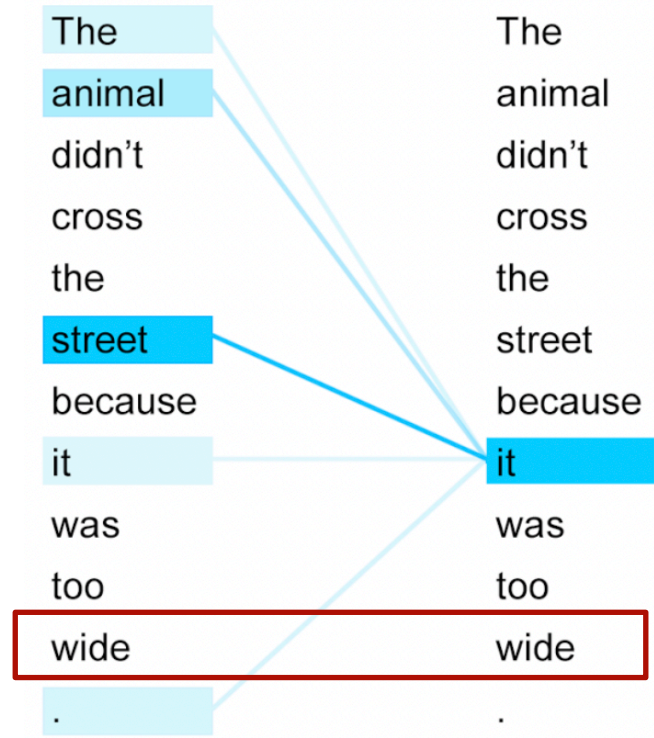
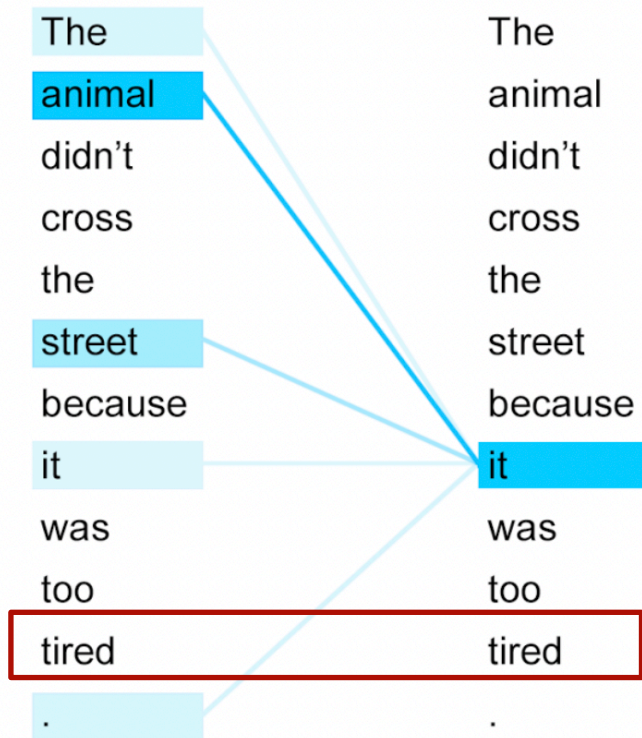
Source input

Transformer Encoder

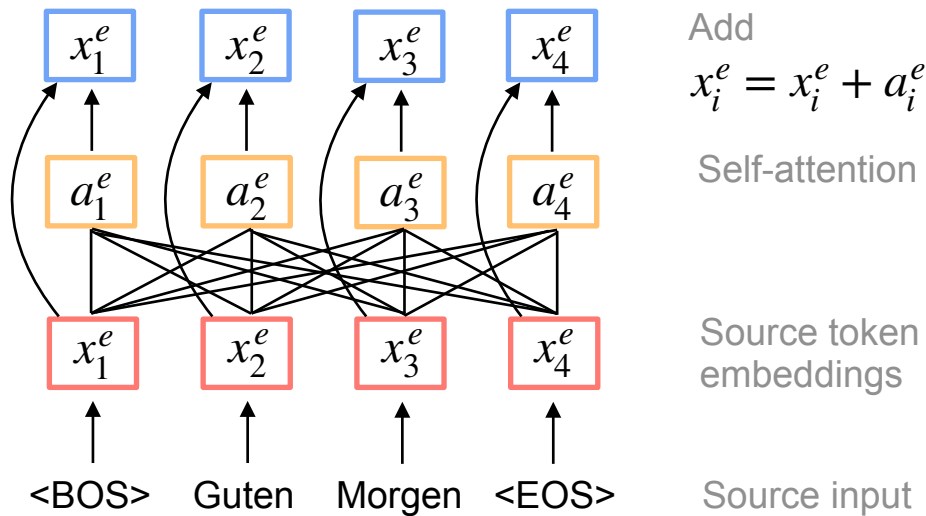
Self-attention weights visualisation



Transformer Encoder

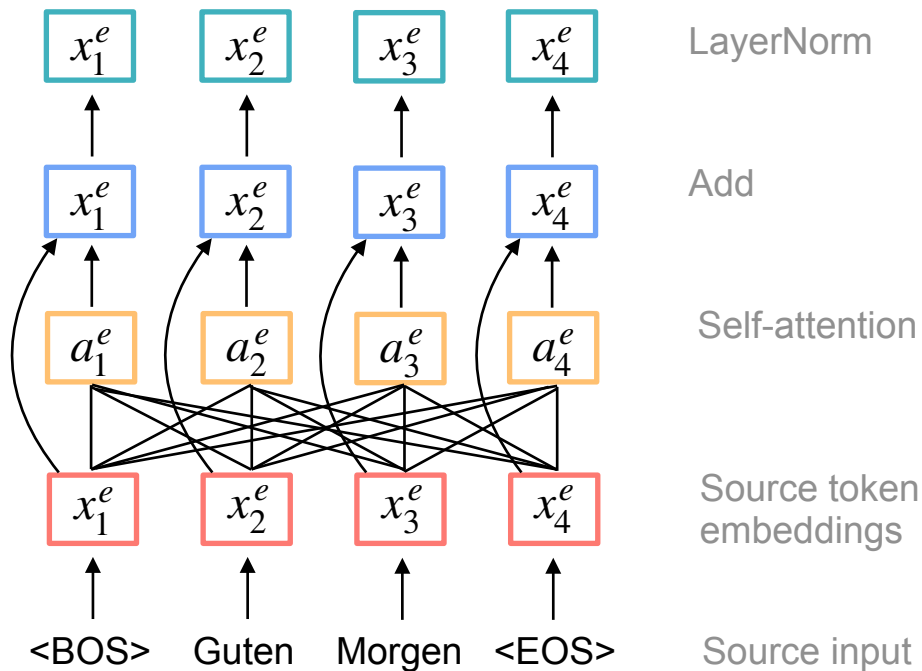


Transformer Encoder



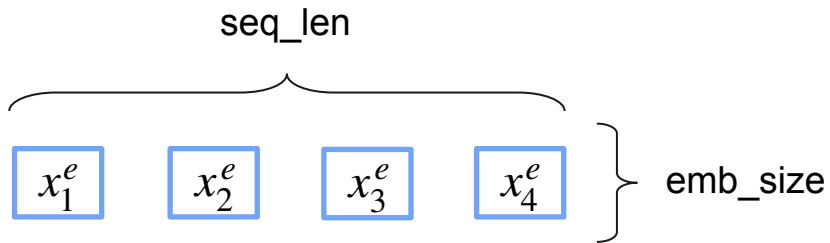
Transformer Encoder

LayerNorm ensures that layer output will always have the same mean and variance, which stabilises neural network training.



LayerNorm

Layer output of size (seq_len, emb_size)



Calculate mean and standard deviation over all elements for each embedding x_i^e :

$$\mu_i = \frac{1}{emb_size} \sum_{j=1}^{emb_size} x_j^e$$

$$\sigma_i = \sqrt{\frac{1}{emb_size} \sum_{j=1}^{emb_size} (x_j^e - \mu_i)^2}$$

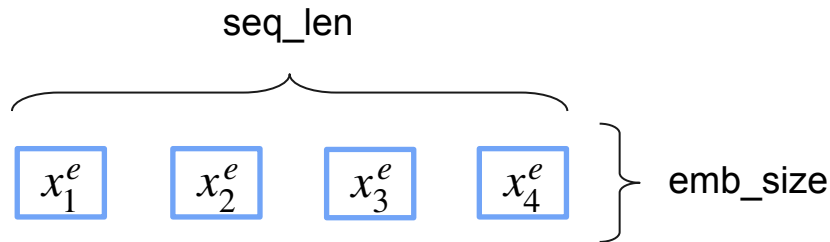
LayerNorm

Calculate mean and standard deviation over all elements for each embedding x_i^e :

$$\mu_i = \frac{1}{emb_size} \sum_{j=1}^{emb_size} x_j^e$$

$$\sigma_i = \sqrt{\frac{1}{emb_size} \sum_{j=1}^{emb_size} (x_j^e - \mu_i)^2}$$

Layer output of size (seq_len, emb_size)



LayerNorm

Calculate mean and standard deviation over all elements for each embedding x_i^e :

$$\mu_i = \frac{1}{emb_size} \sum_{j=1}^{emb_size} x_j^e$$

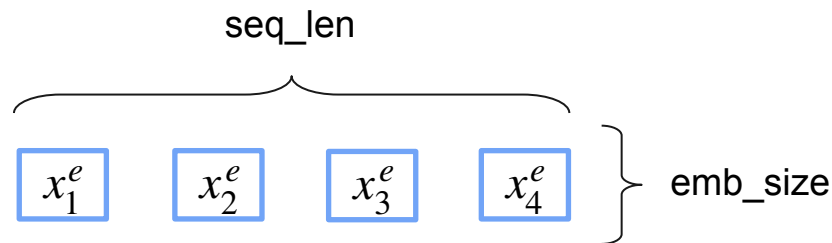
$$\sigma_i = \sqrt{\frac{1}{emb_size} \sum_{j=1}^{emb_size} (x_j^e - \mu_i)^2}$$

After that we update x_i^e as follows:

$$x_i^e = \frac{x_i^e - \mu_i}{\sigma_i + \epsilon} \cdot \gamma + \beta$$

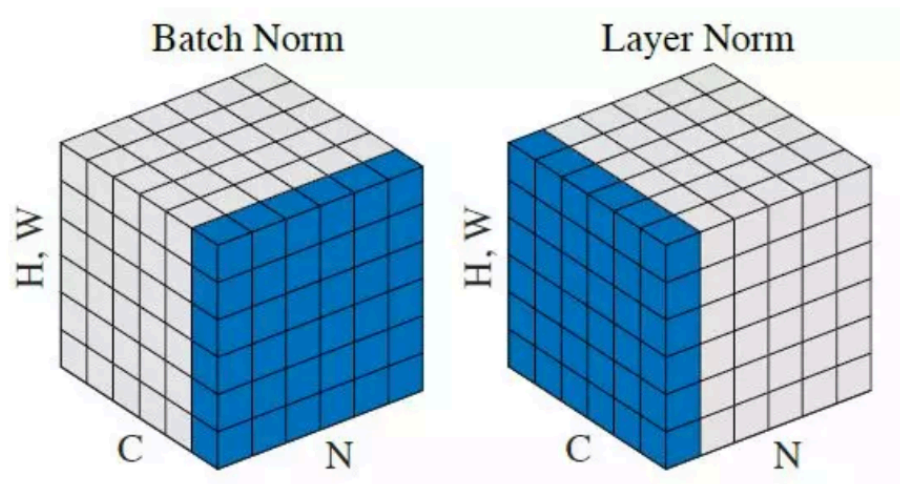
Trainable parameters

Layer output of size (seq_len, emb_size)

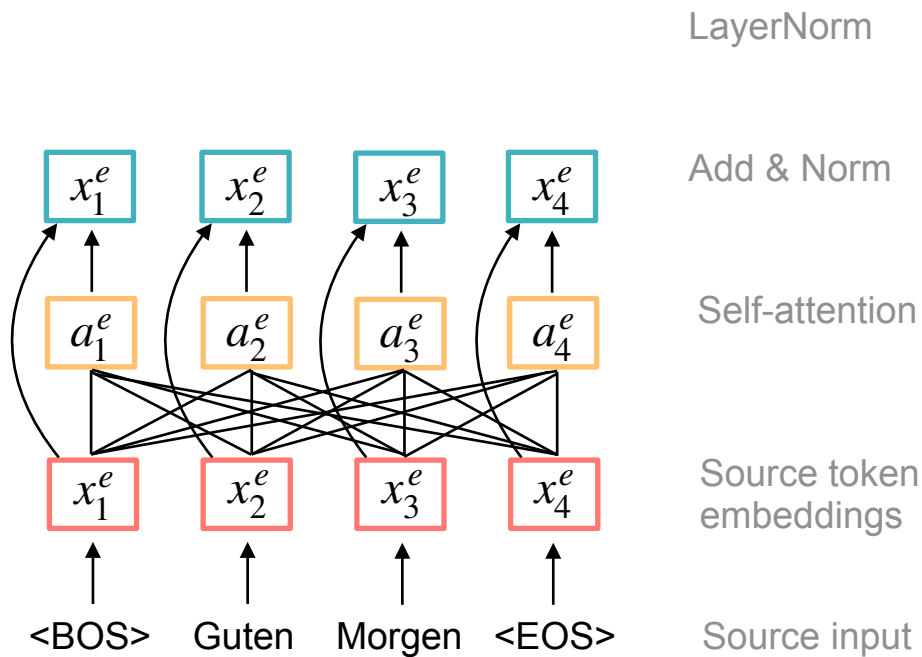


LayerNorm

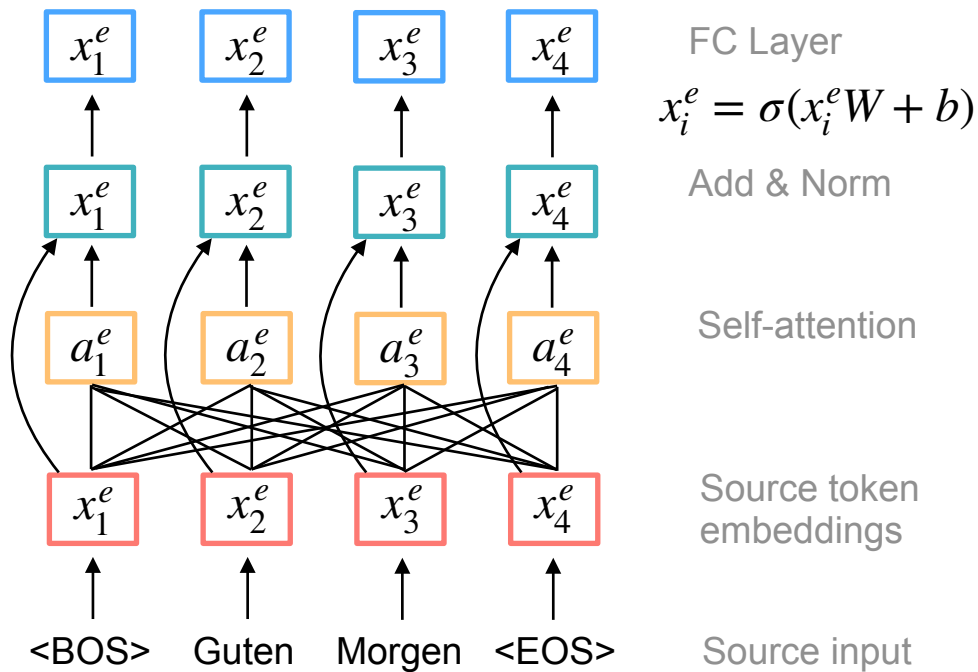
If you are familiar with BatchNorm, this picture might help understand LayerNorm:



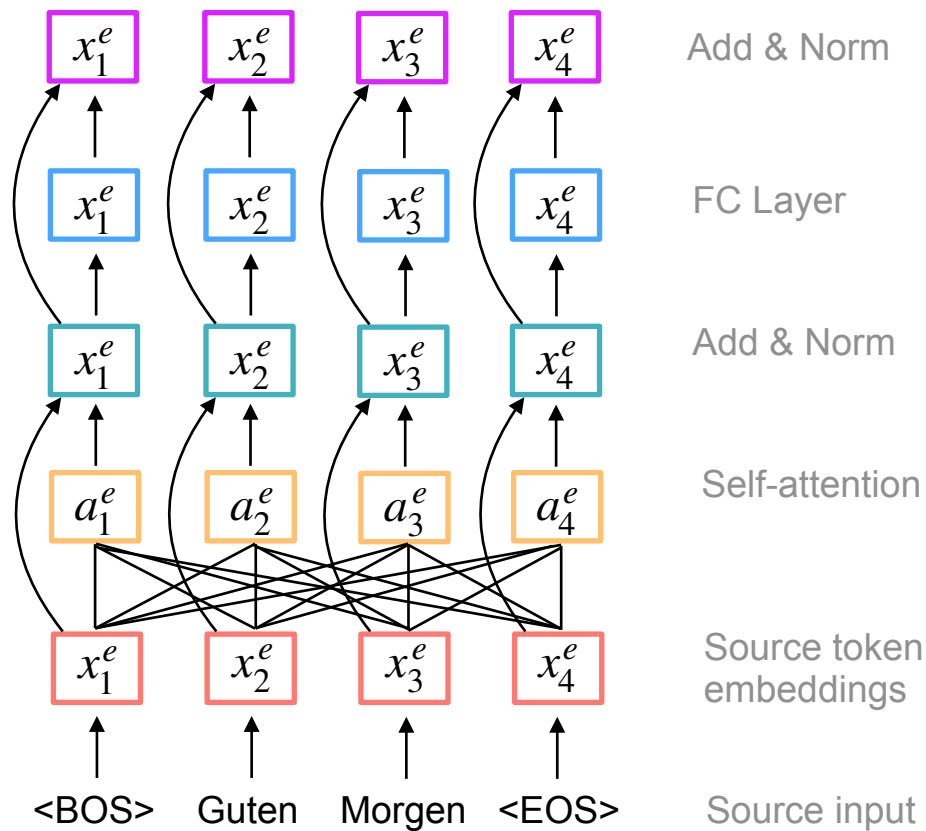
Transformer Encoder



Transformer Encoder



Transformer Encoder



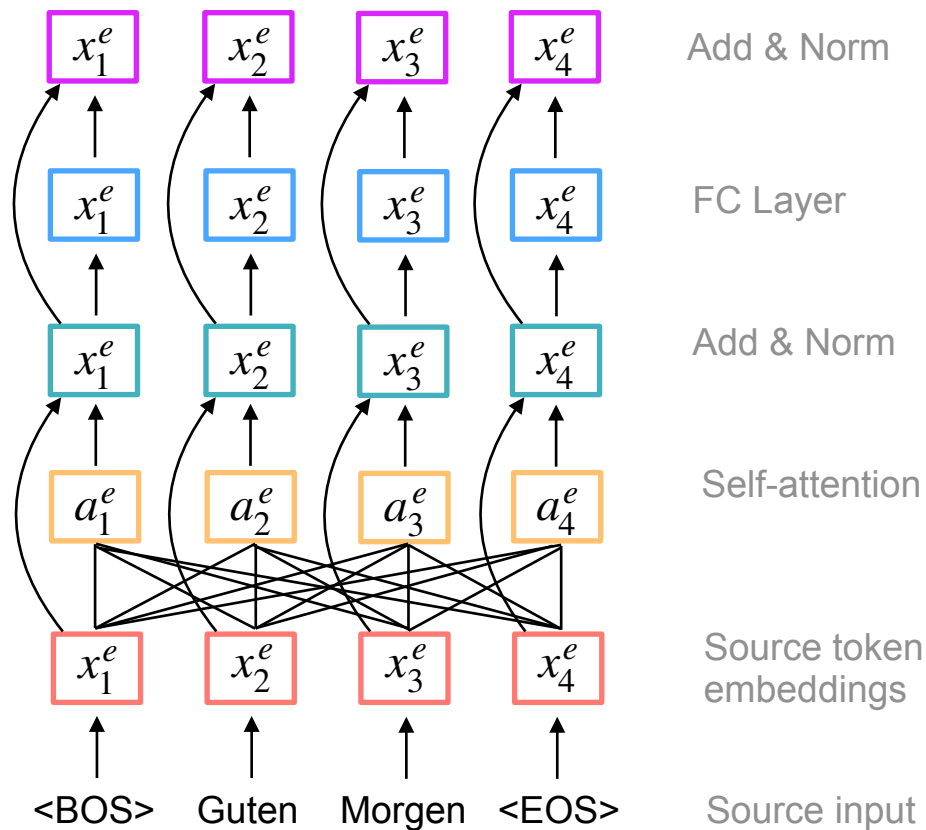
Transformer Encoder

Result of Encoder Block is updated token embeddings

Each layer processes all the embeddings in parallel.

Each layer except self-attention processes embeddings independently.

Transformer Encoder can process inputs of arbitrary size

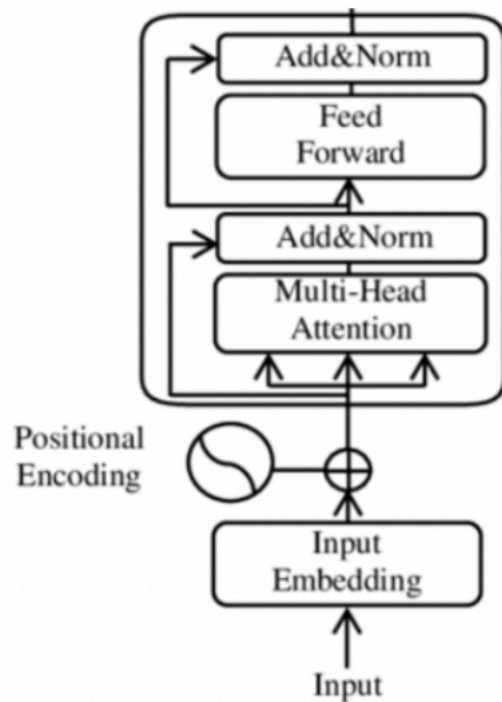


Transformer Encoder

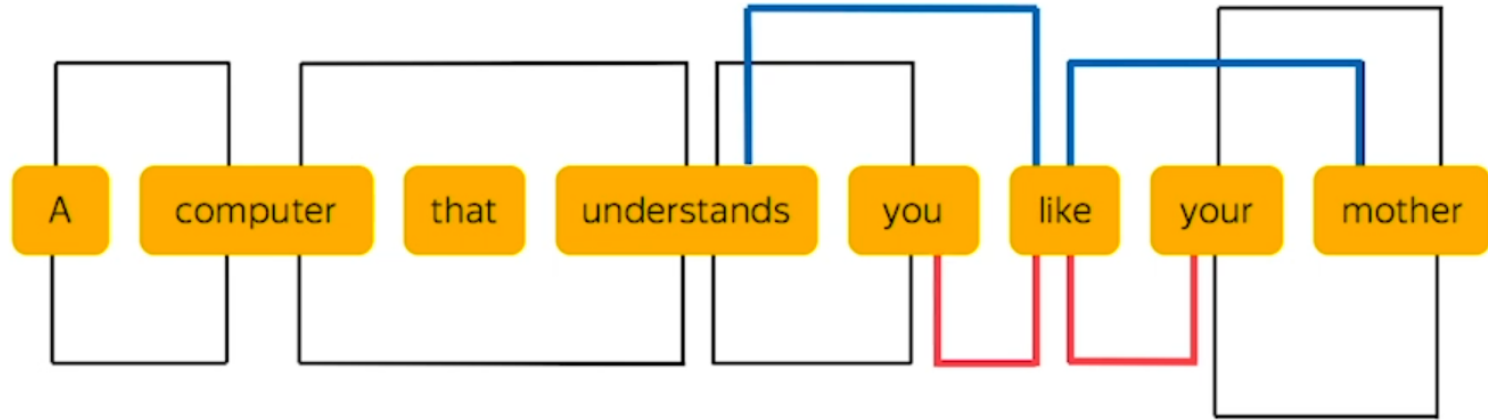
We have almost everything about Encoder.

What's left:

- Multi-head attention
- Positional Encoding



Multi-Head Self-Attention

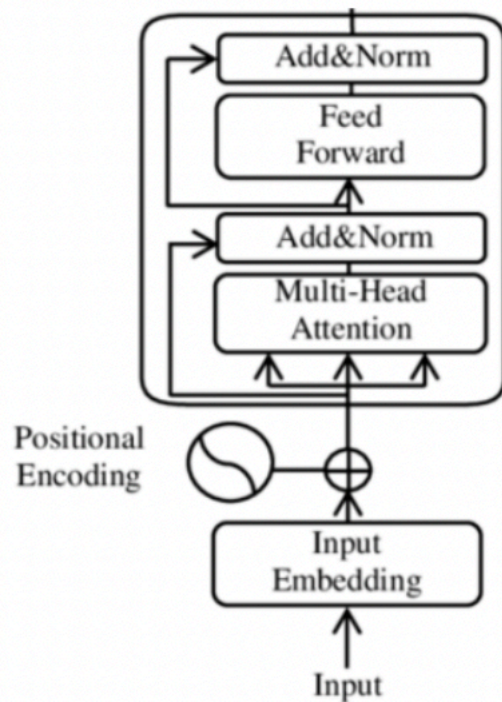


Transformer Encoder

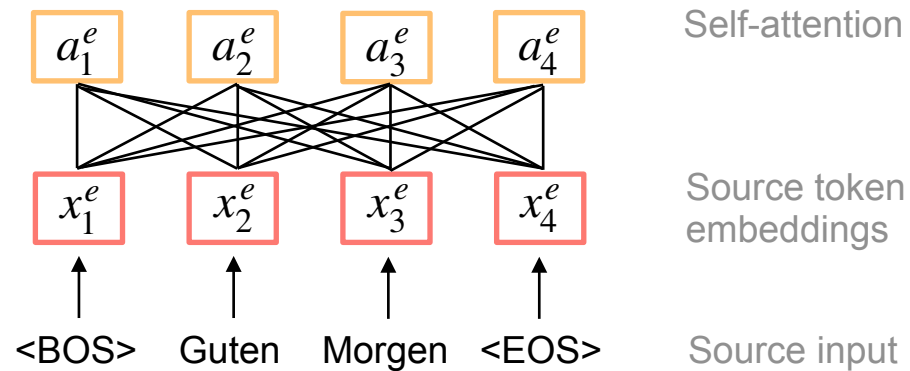
We have almost everything about Encoder.

What's left:

- Multi-head attention
- Positional Encoding



Multi-Head Self-Attention



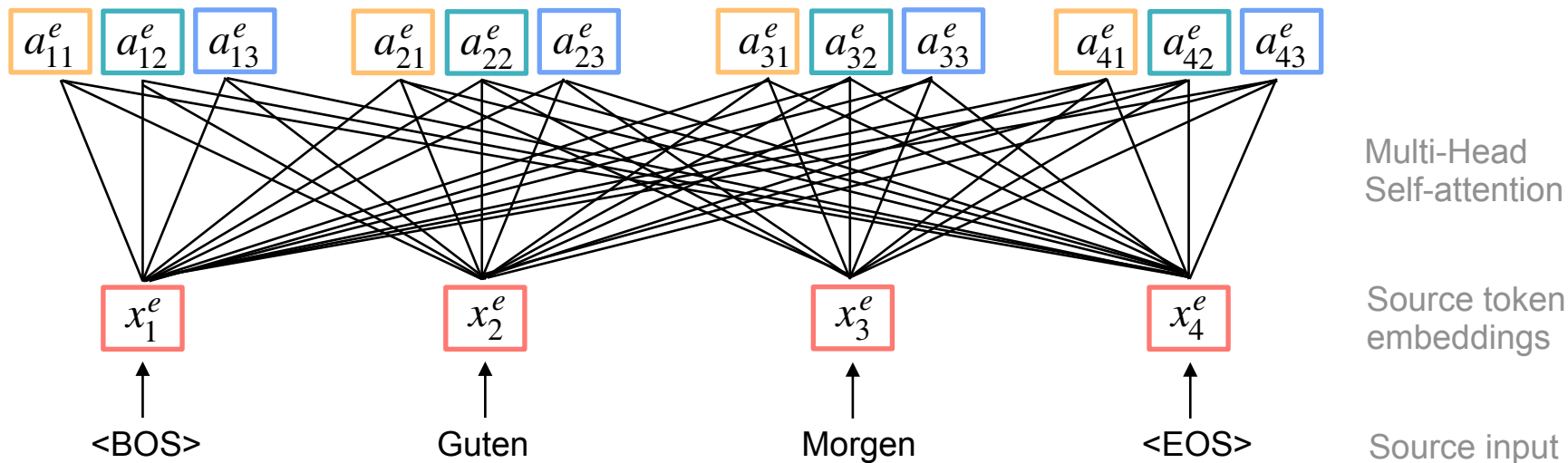
Multi-Head Self-Attention

All attention heads have
different trainable parameters

k is an index of attention head:

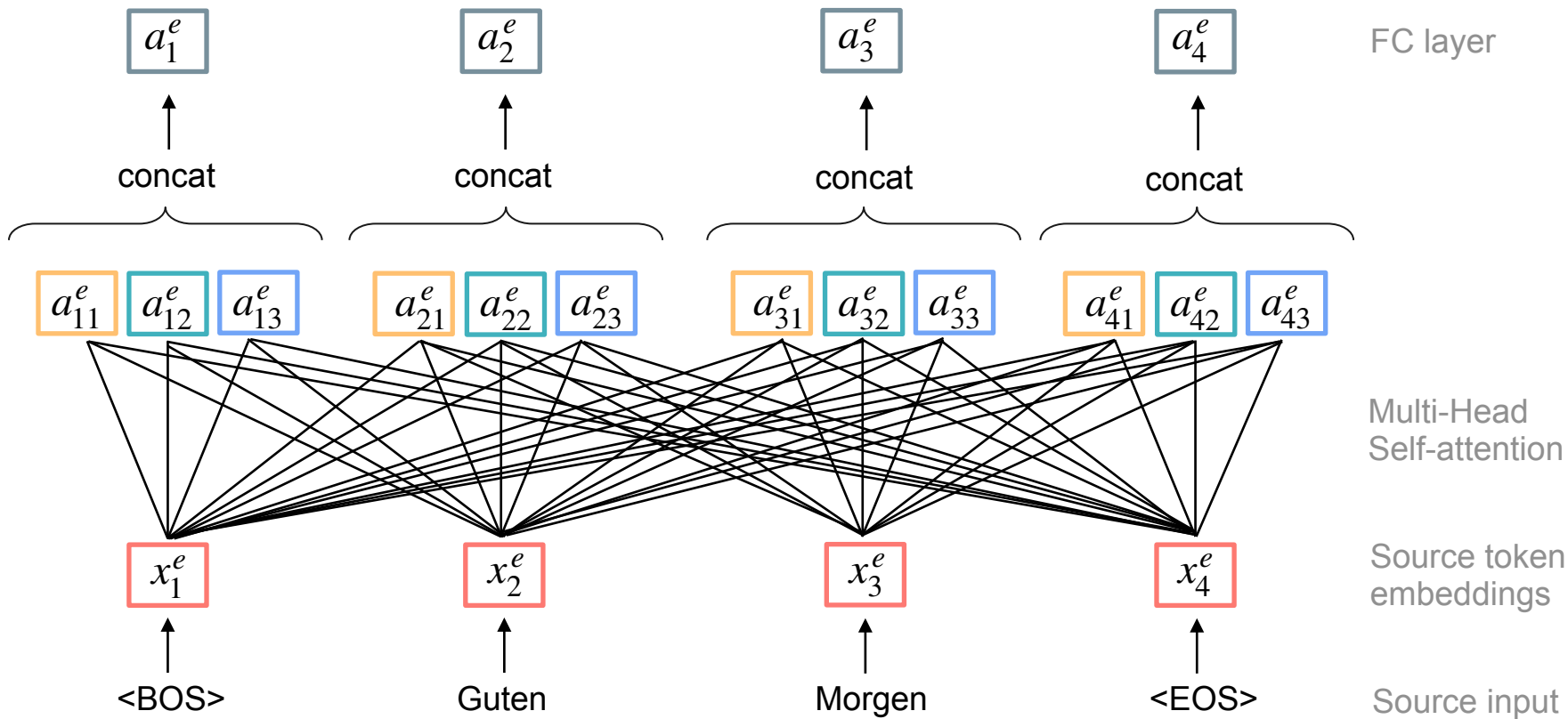
$$w_{ij}^k = s^k(x_i^e, x_j^e)$$

$$a_{ki}^e = \sum w_{ij}^k x_j^e$$



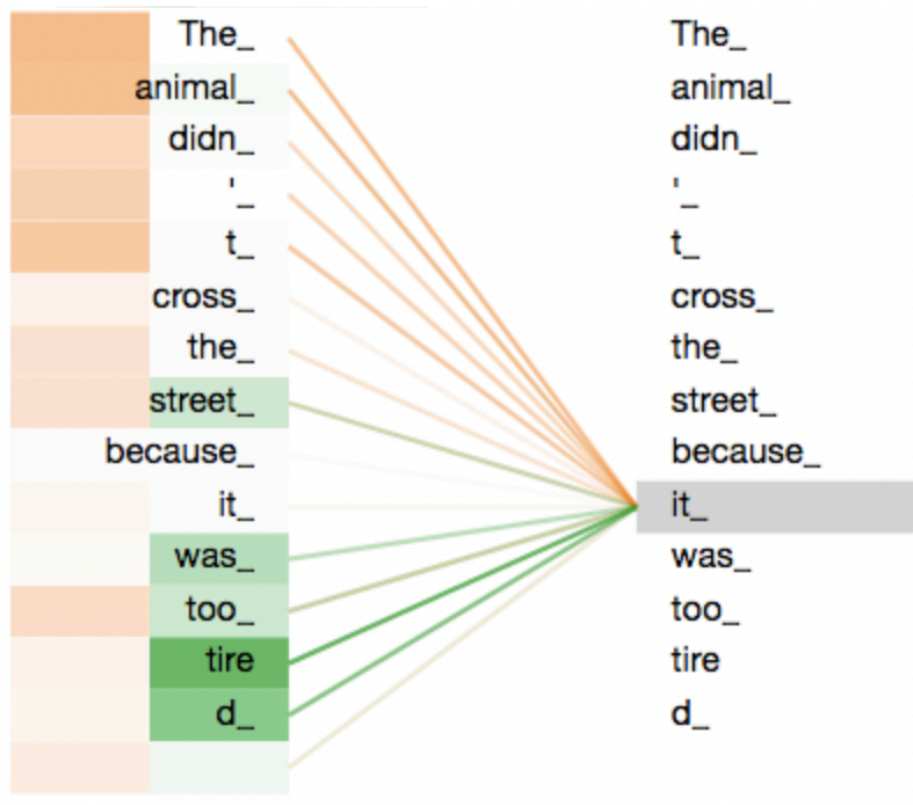
Multi-Head Self-Attention

$$a_i^e = \sigma([a_{i1}^e, a_{i2}^e, a_{i3}^e]W + b)$$



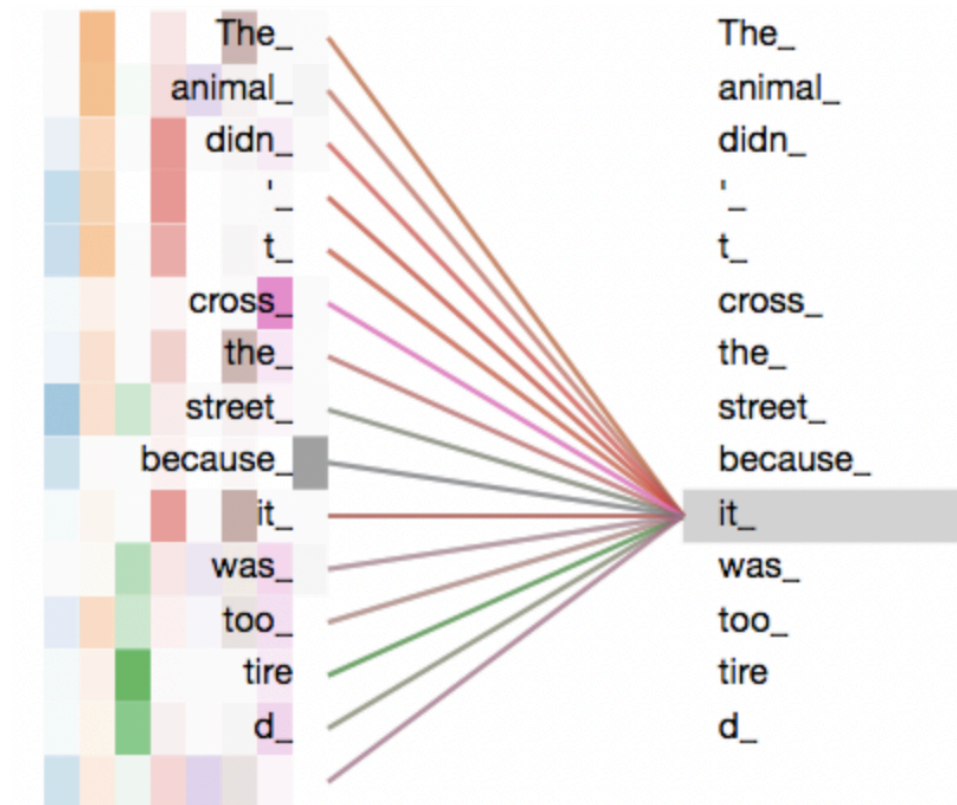
Multi-Head Self-Attention

Different attention heads attend to different tokens, i.e. extract different information from the sentence



Multi-Head Self-Attention

Different attention heads attend to different tokens, i.e. extract different information from the sentence

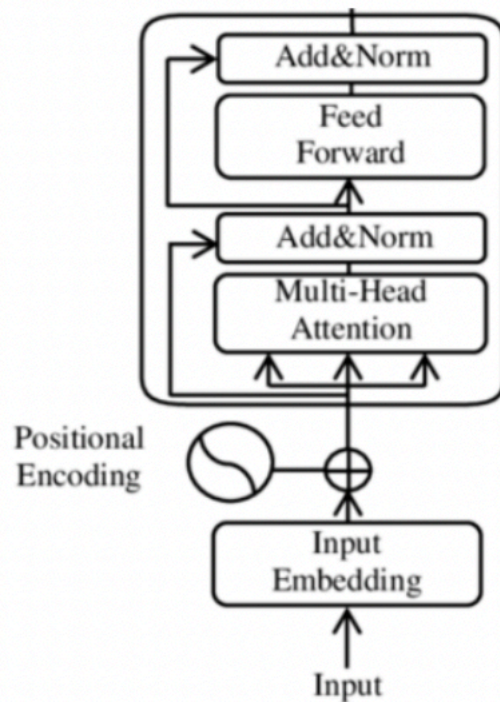


Transformer Encoder

We have almost everything about Encoder.

What's left:

- Positional Encoding

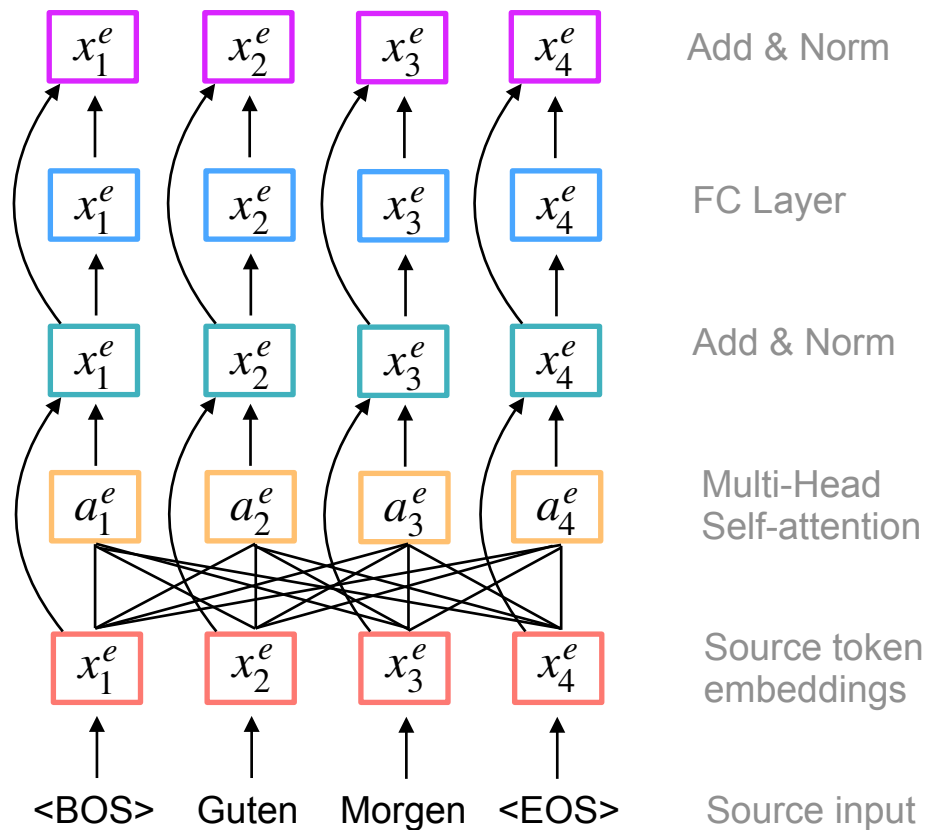


Positional Encoding

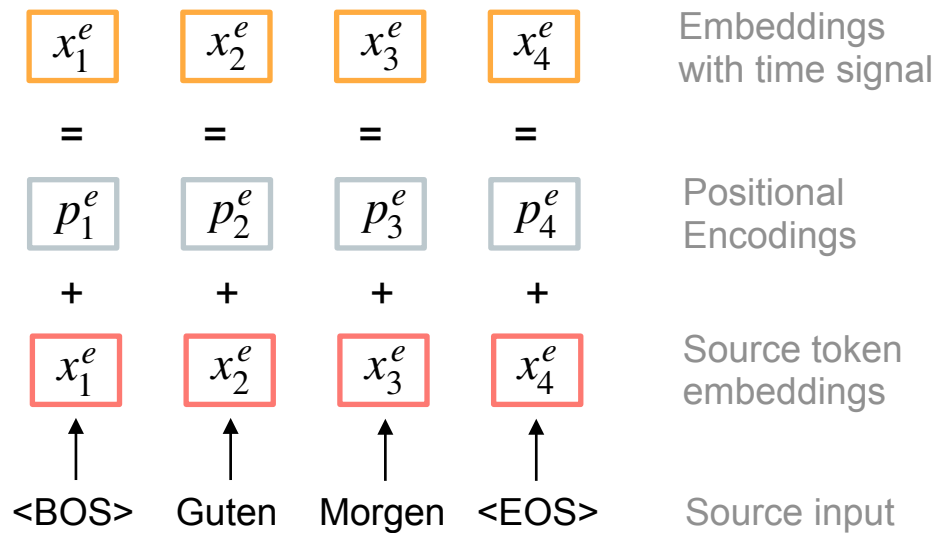
In our network there's no information about token positions in the sentence

But it is important for updating each token's embeddings:

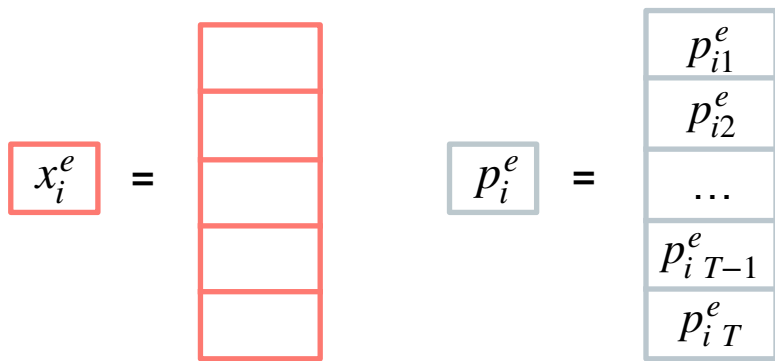
- cat runs horse
- horse runs cat



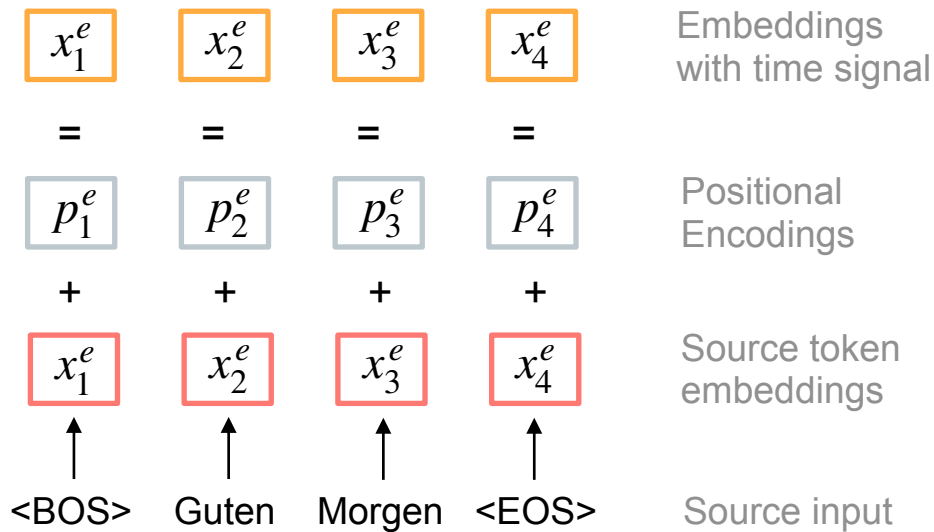
Positional Encoding



Positional Encoding



$$p_{it}^e = \begin{cases} \sin\left(\frac{t}{10000^{\frac{2k}{emb_dim}}}\right), & \text{if } i = 2k \\ \cos\left(\frac{t}{10000^{\frac{2k}{emb_dim}}}\right), & \text{if } i = 2k + 1 \end{cases}$$



Positional Encoding

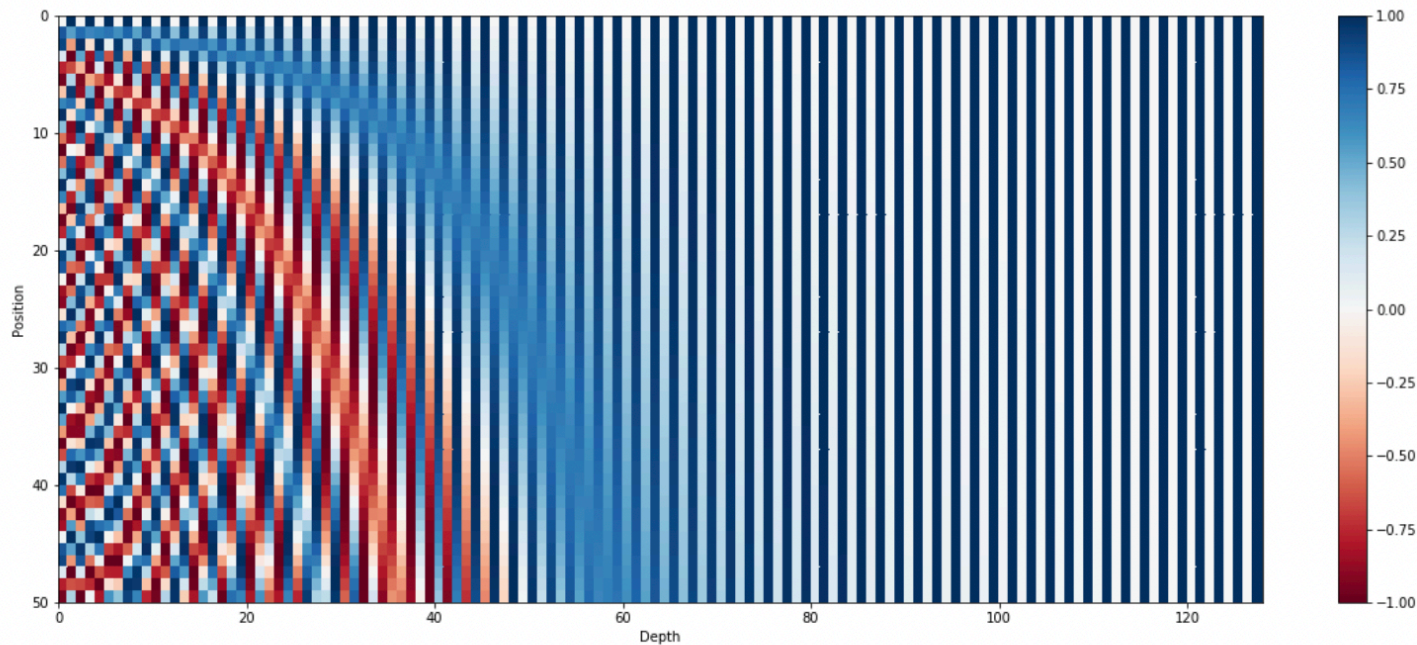


Figure 2 - The 128-dimensional positional encoding for a sentence with the maximum length of 50. Each row represents the embedding vector $\vec{p}_t$

Positional Encoding

Why choose such a function:

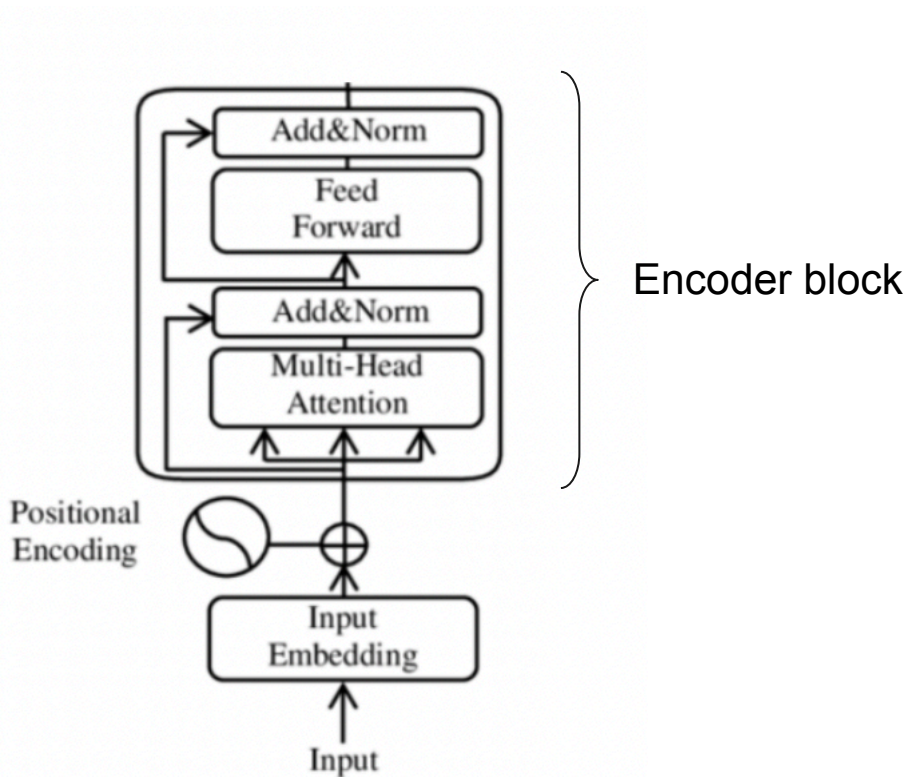
In this work, we use sine and cosine functions of different frequencies:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k , PE_{pos+k} can be represented as a linear function of PE_{pos} .

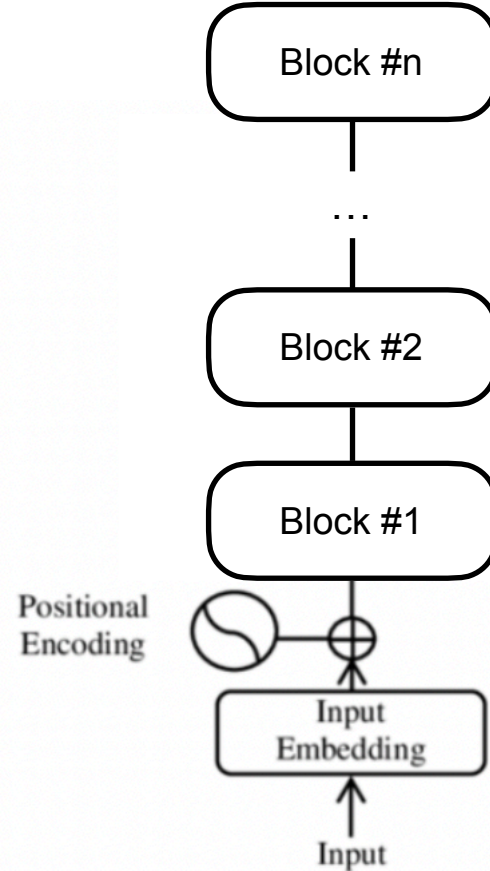
Transformer Encoder

We've built a Transformer block!

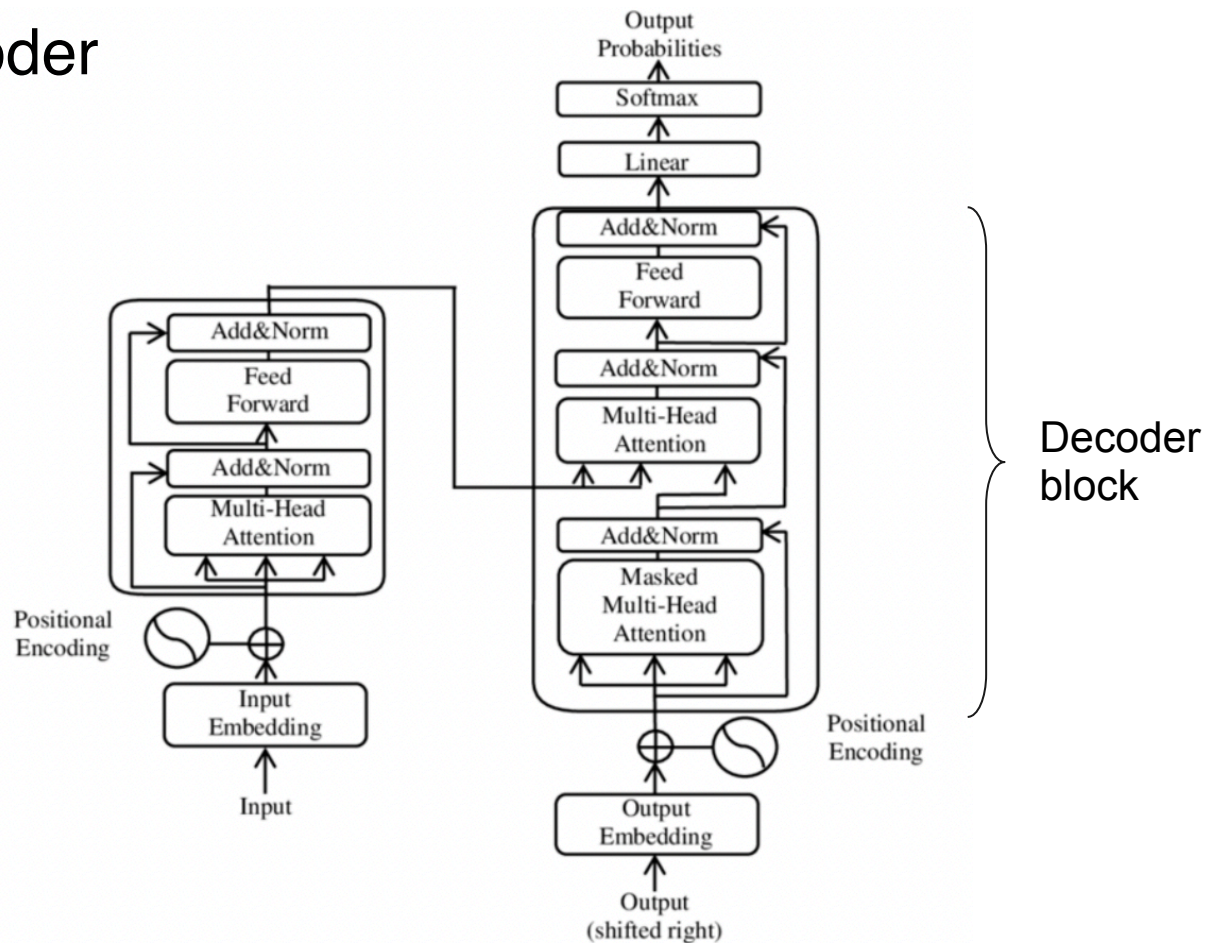


Transformer Encoder

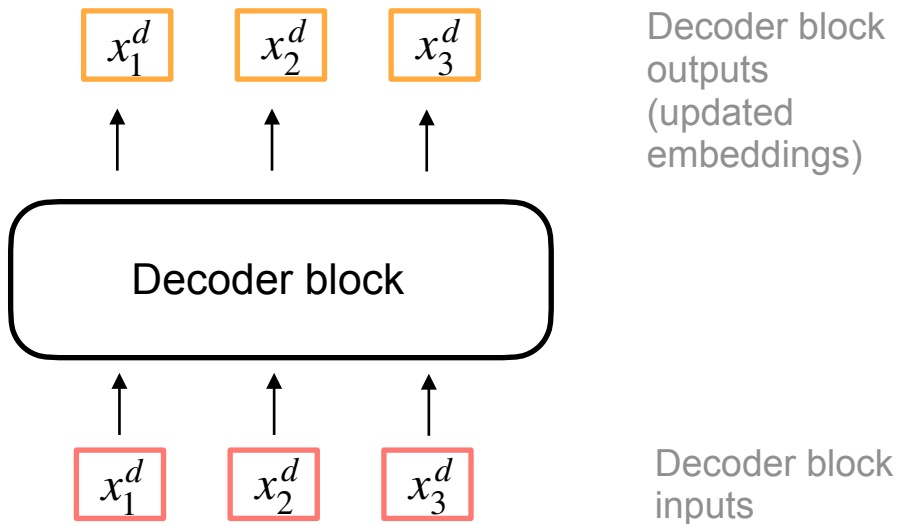
Encoder can have multiple blocks stacked together



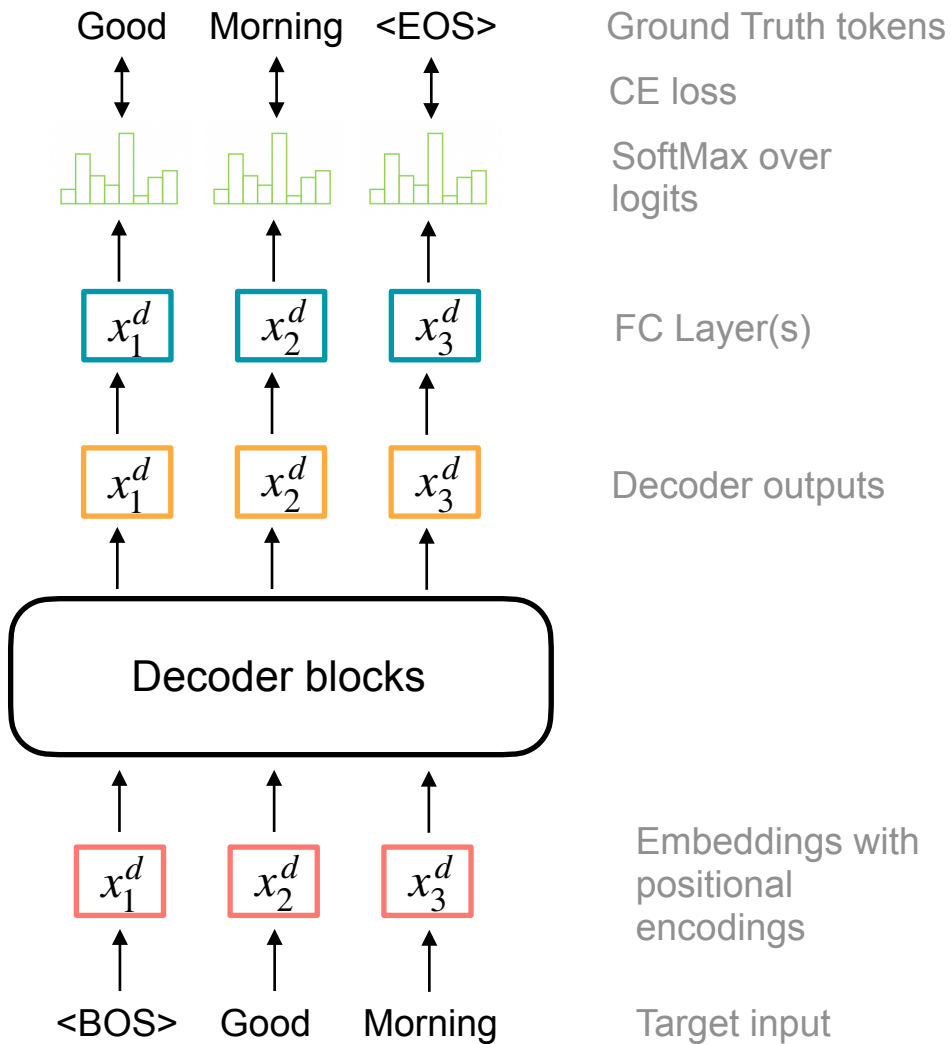
Transformer Decoder



Transformer Decoder

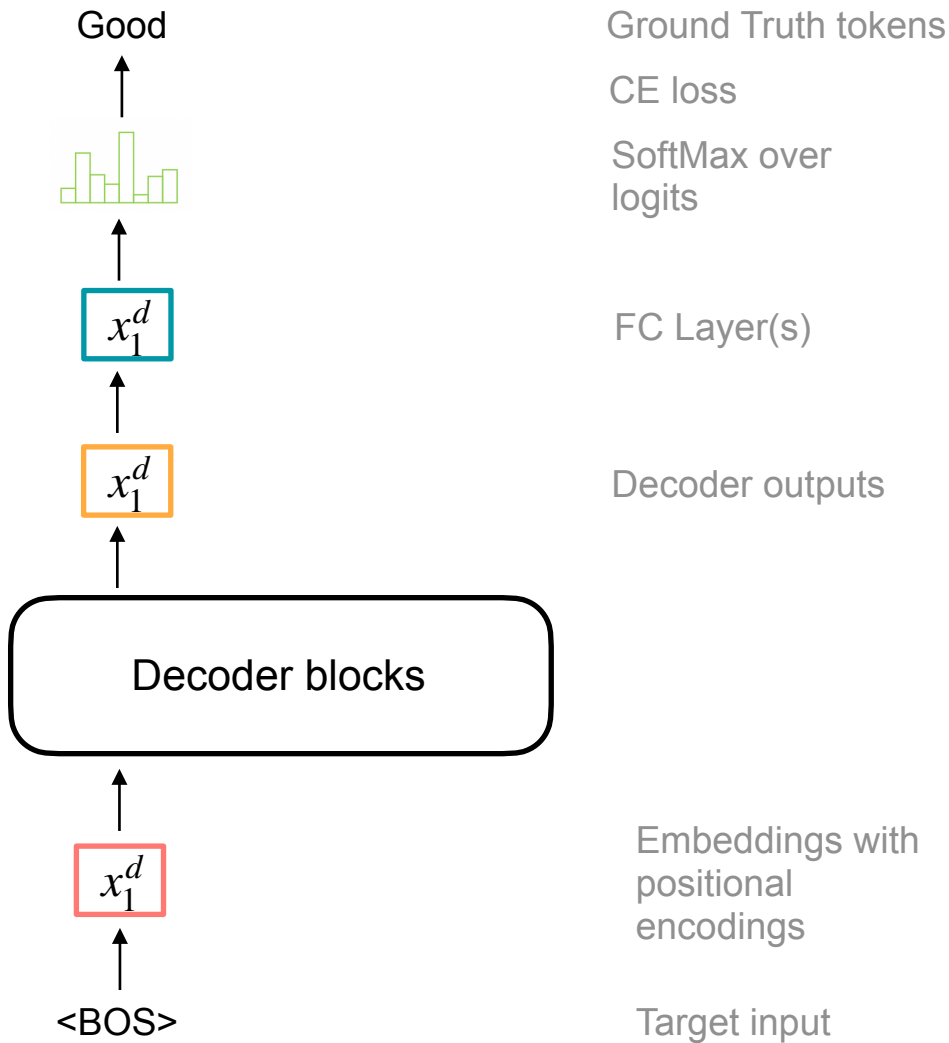


Transformer Decoder



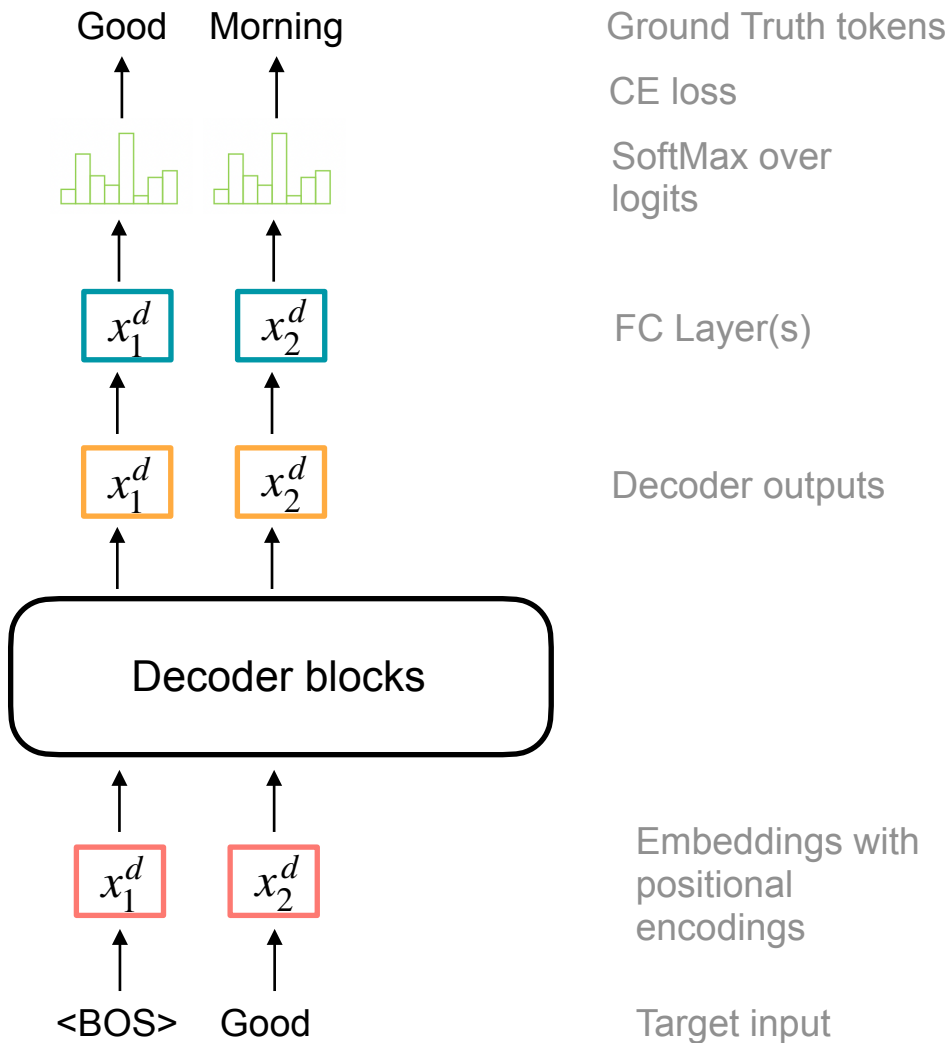
Transformer Decoder

During inference decoder
generates output sequence
autoregressively



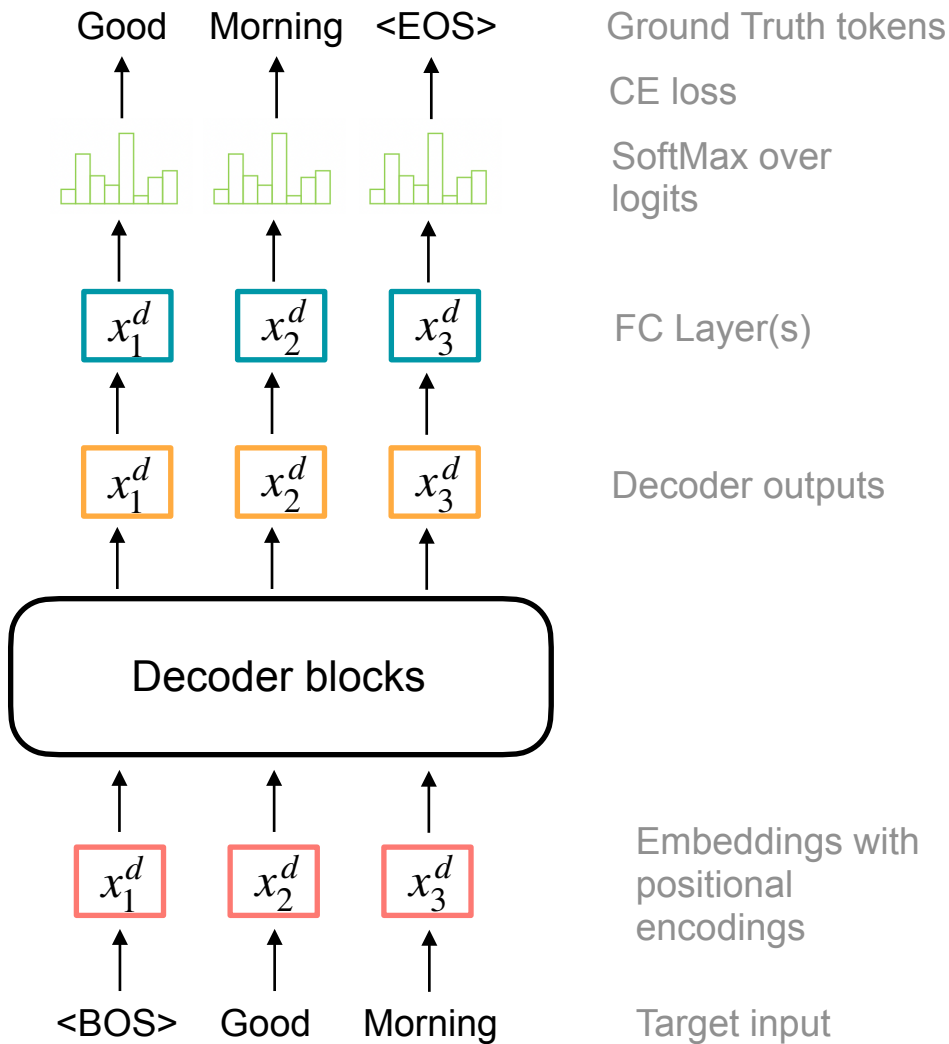
Transformer Decoder

During inference decoder
generates output sequence
autoregressively

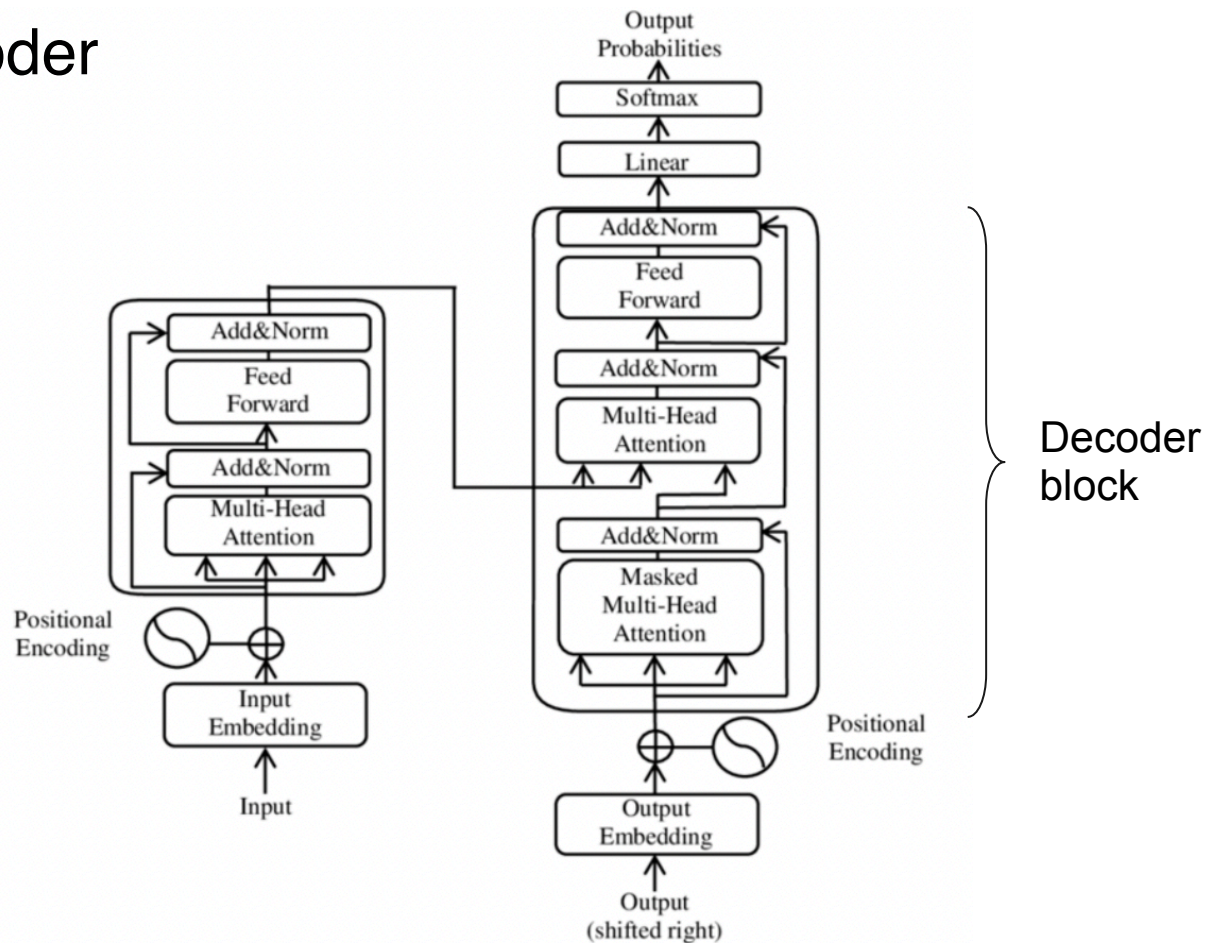


Transformer Decoder

During inference decoder
generates output sequence
autoregressively



Transformer Decoder

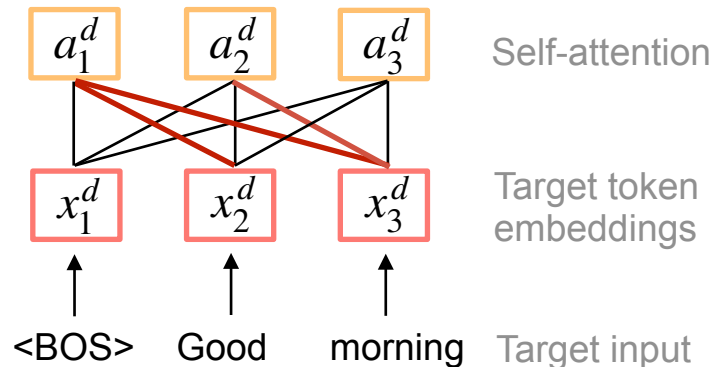
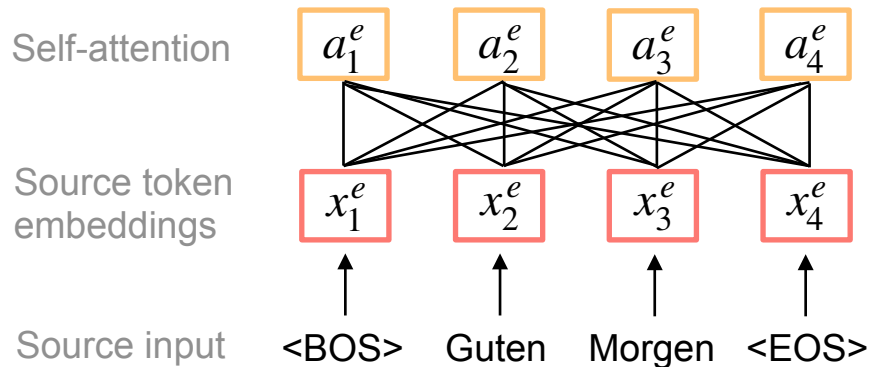


Masked Self-Attention

During training *embedding x_i^d cannot attend to future embeddings*

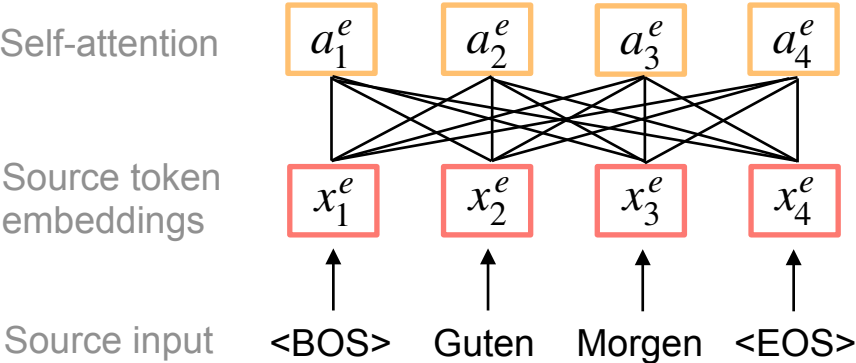
Encoder

Decoder

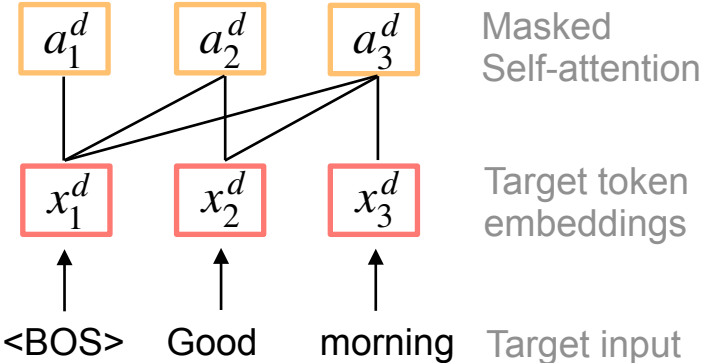


Masked Self-Attention

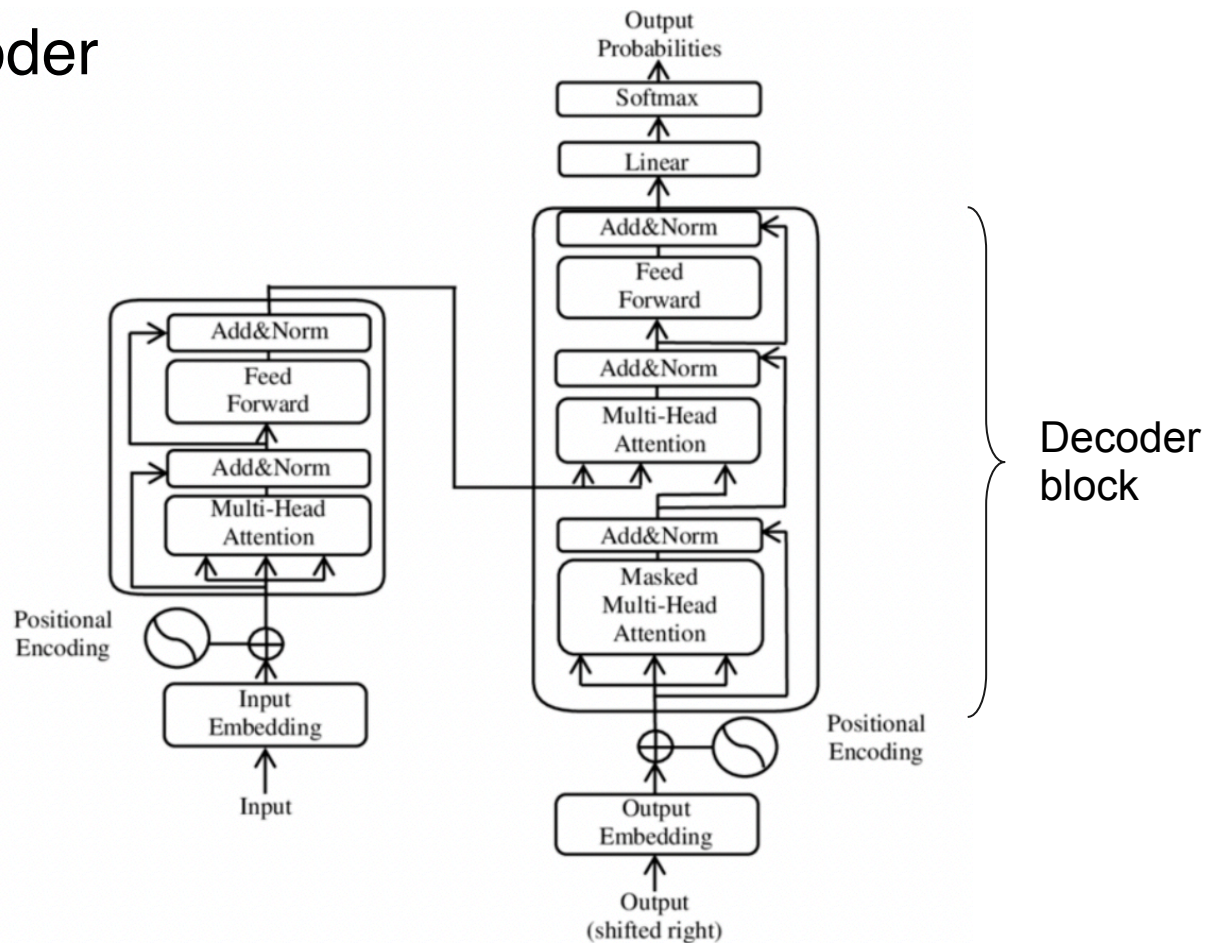
Encoder



Decoder



Transformer Decoder



$$a_1^{ed} = \sum w_{1i} * x_i^e$$

$$a_1^e$$

0.20

0.69

0.03

0.08



w_{11}

w_{12}

w_{13}

w_{14}

$$w_{1i} = s(x_1^d, x_i^e)$$

Final Encoder output

x_1^e

x_2^e

x_3^e

x_4^e

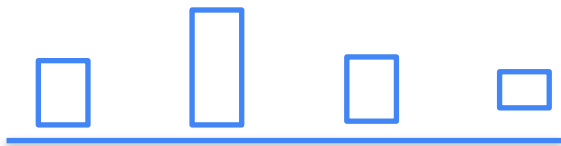
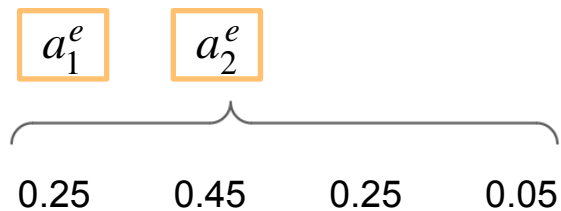
x_1^d

x_2^d

x_3^d

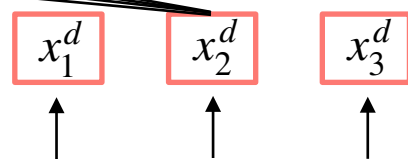
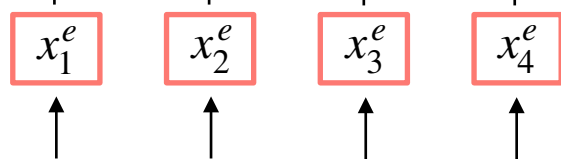
Decoder layer output

$$a_2^{ed} = \sum w_{2i} * x_i^e$$



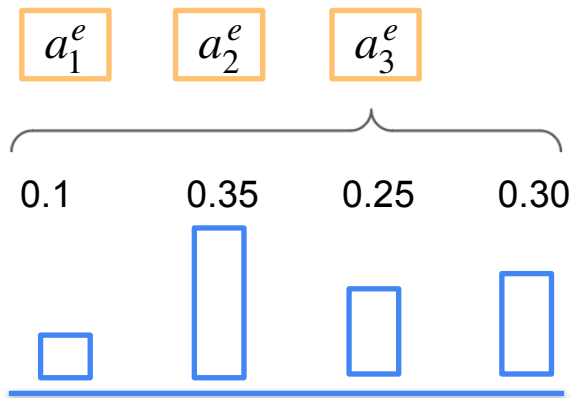
$$w_{2i} = s(x_2^d, x_i^e)$$

Final Encoder output



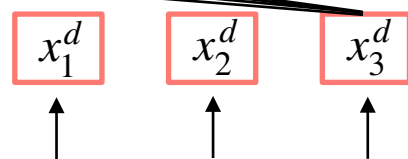
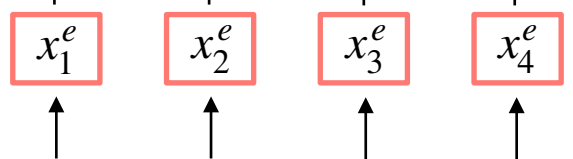
Decoder layer output

$$a_3^{ed} = \sum w_{3i} * x_i^e$$



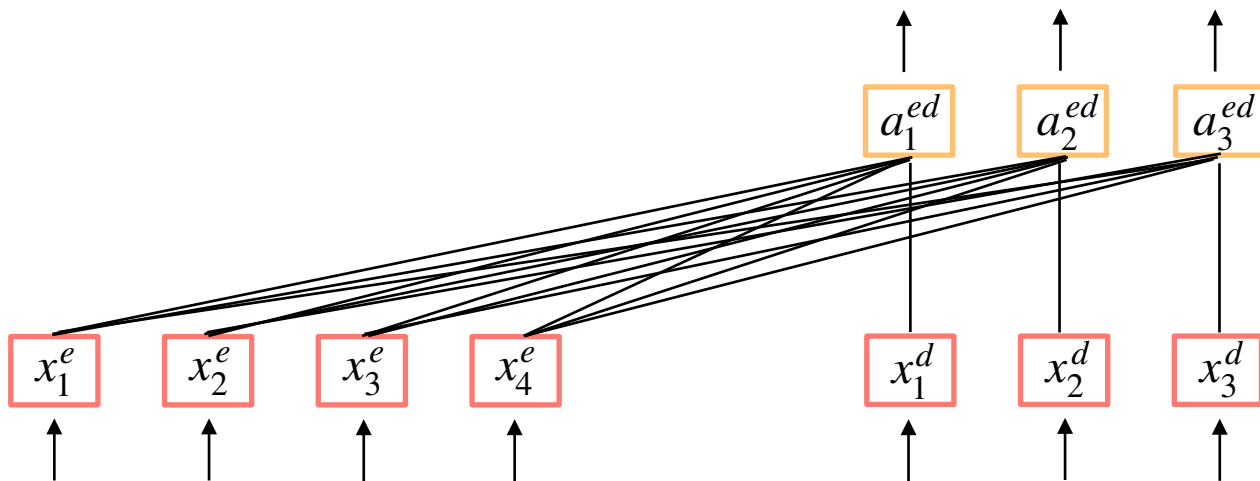
$$w_{3i} = s(x_3^d, x_i^e)$$

Final Encoder output



Decoder layer output

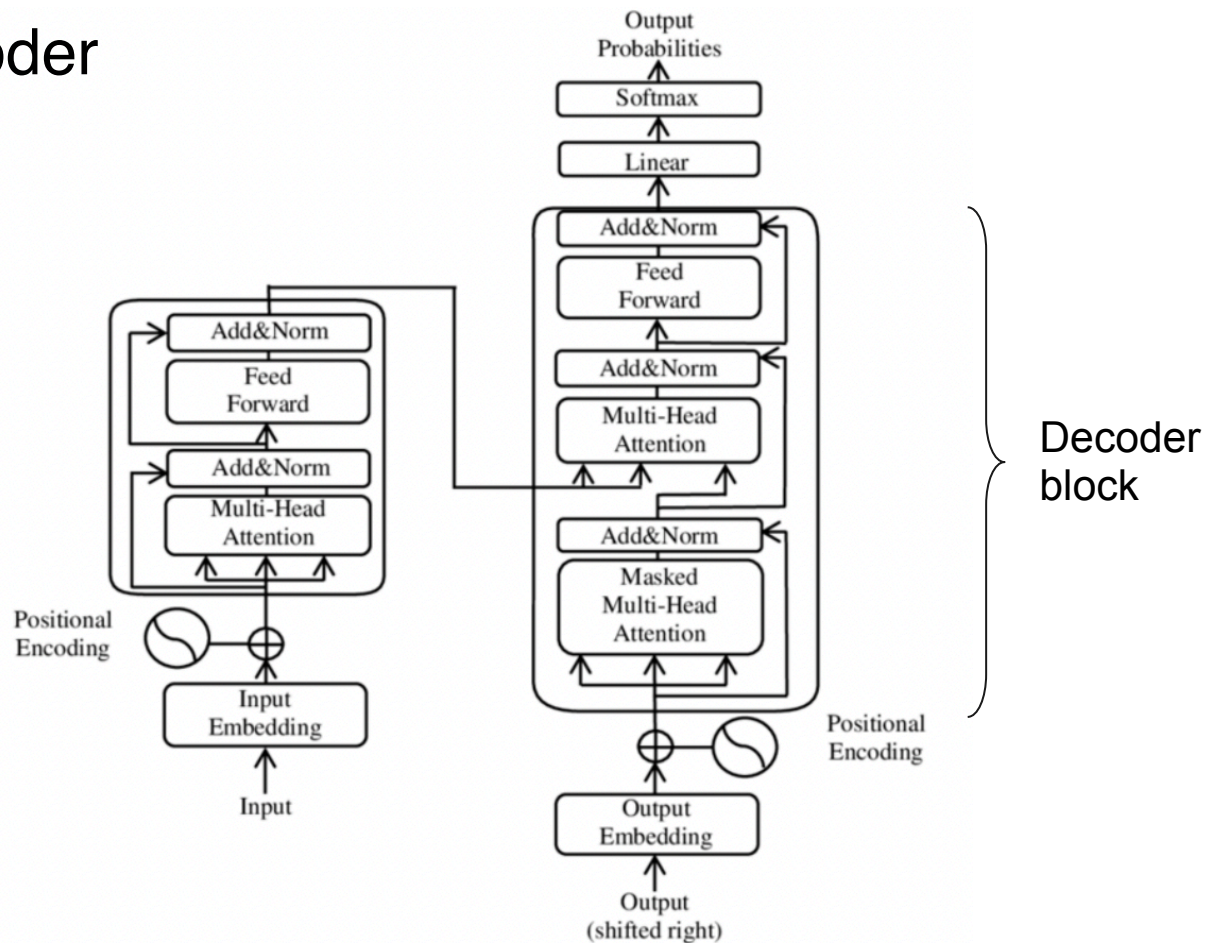
Final Encoder
output



Cross-Attention

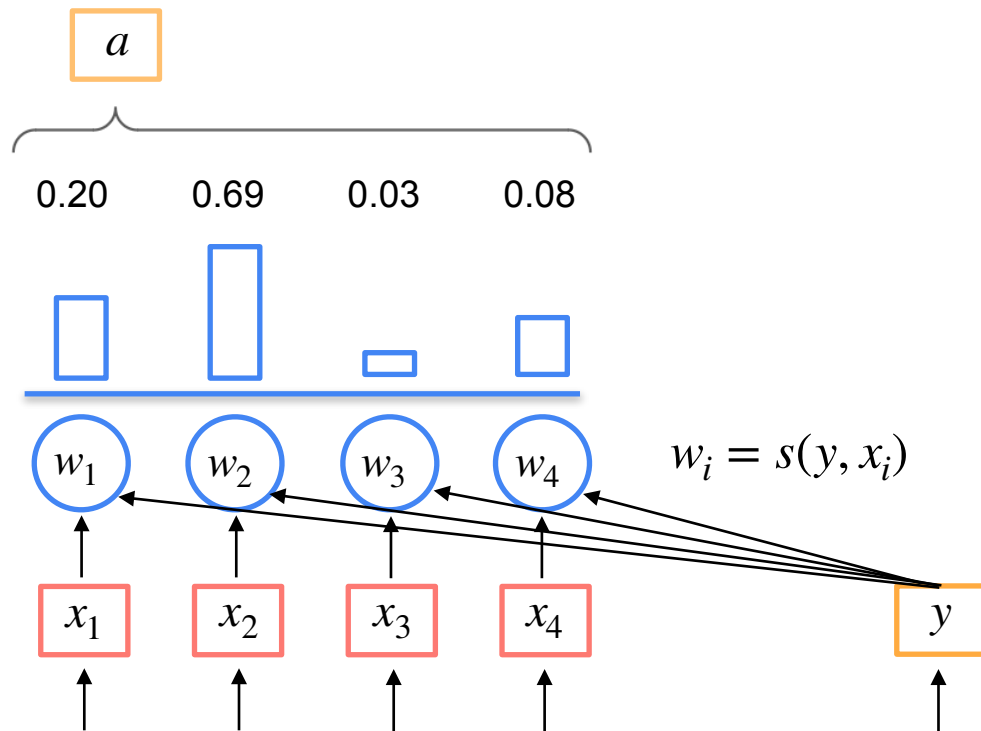
Decoder layer
output

Transformer Decoder

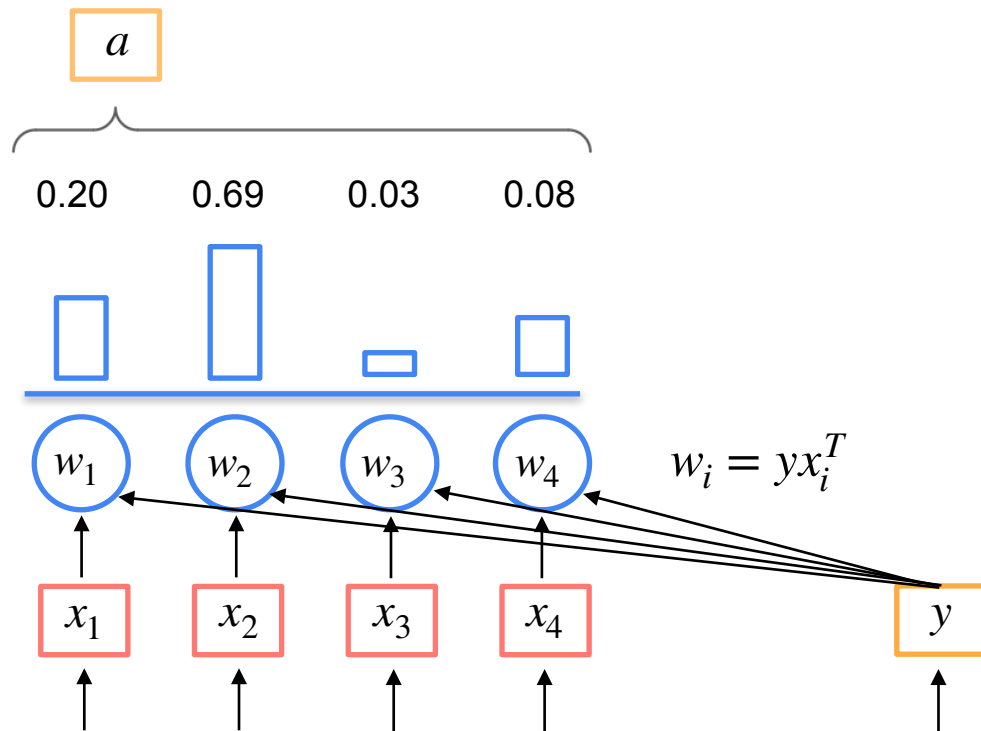


QKV Attention

$$a = \sum w_i * x_i$$



$$a = \sum w_i * x_i$$

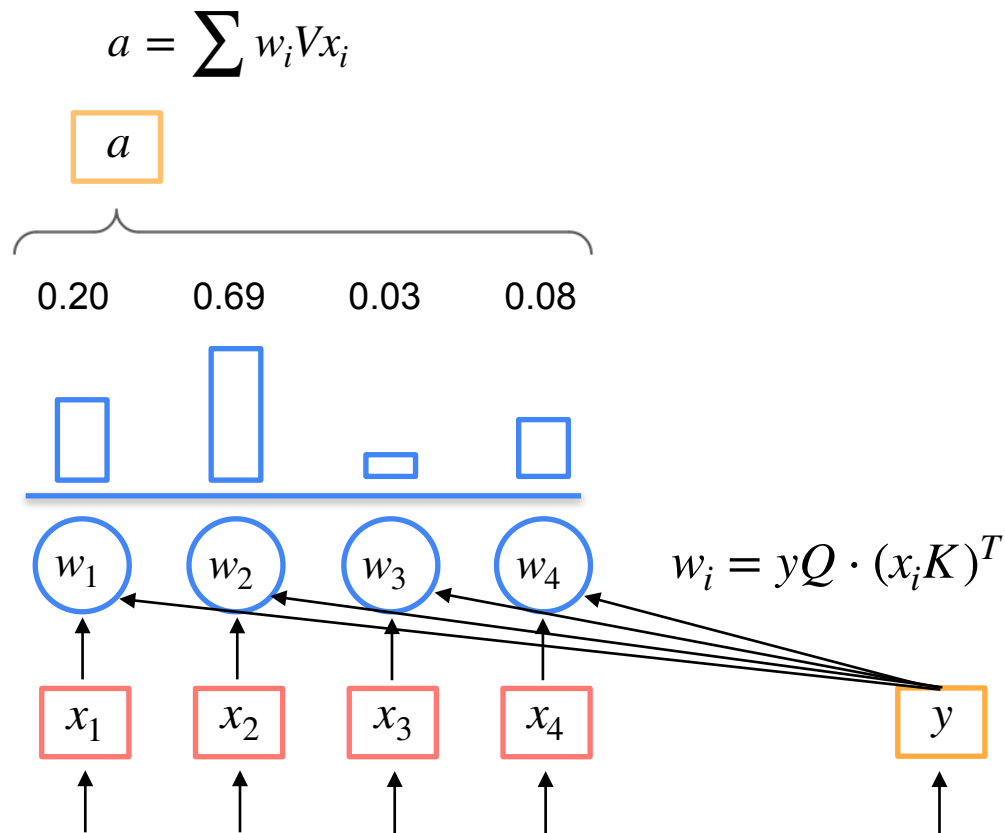


Attention layer has 3 trainable matrices:

- $Q_{d \times m}$ (query)
- $K_{d \times m}$ (key)
- $V_{d \times m}$ (value)

Here:

- d is a length of embeddings;
- m can be chosen arbitrarily

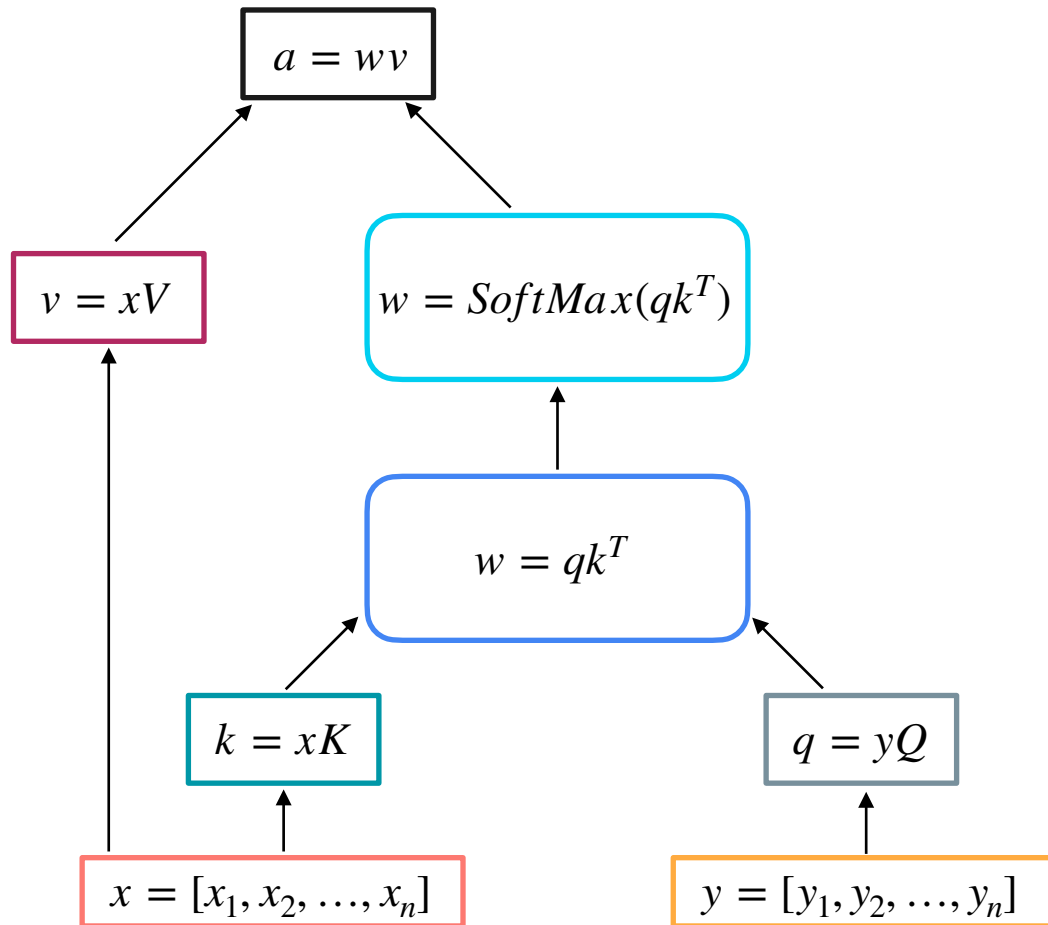


Attention layer has 3 trainable matrices:

- $Q_{d \times m}$ (query)
- $K_{d \times m}$ (key)
- $V_{d \times m}$ (value)

$$w = \text{SoftMax}\left(\frac{xK(yQ)^T}{\sqrt{m}}\right)$$

$$a = w(xV)$$



Transformer Decoder

